

claude-windows / df5d5fb2-534a-447a-8181-b222f94655f3

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\df5d5fb2-534a-447a-8181-b222f94655f3\subagents\workflows\wf_133acdee-dab\agent-ab186f3f099abed5d.jsonl`  
- SHA-256: `68a4f0979e4279f993762d304222559ba38533a04e1d459e09935782a41d5a3`  
- Source modified: `2026-07-06T17:30:18+00:00`  
- Imported at: `2026-07-08T15:59:35+00:00`  
- project: `wf_133acdee-dab`  
- session_id: `df5d5fb2-534a-447a-8181-b222f94655f3`
```

Transcript

1. user (2026-07-06T17:26:48.058Z)

You are a CONTROL-PLANE INTERACTION reviewer.

REPO ROOT: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer

DIFF UNDER REVIEW (read this first): C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-

indexer/df5d5fb2-534a-447a-8181-b222f94655f3/scratchpad/wf294_fix.diff

GOAL OF THE CHANGE (wf294 incident): the Cloud Run rally-worker reported a run as status="success" with 0 segments even when the WASB shuttle-trajectory detector TIMED OUT (1800s) and produced zero trajectory. A detector timeout/abort was indistinguishable from a legitimately 0-rally video. The fix must make "detector failed/timed out" surface as a failed status (NOT "success") in report.json AND the completion marker, while a genuine 0-rally match stays a success.

WHAT THE FIX DID (4 files, all in the diff):

1. backend/pipeline/segmenters/trajectory_hybrid.py ? process_video() now returns a backend.results.Failure (instead of a bare []) on TWO detector-failure paths: the env healthcheck-not-ready path, and the "run_predict produced no trajectory" abort path. A trajectory-produced-but-zero-windows case still returns bare [] (a legit 0-rally). Return annotation widened to Union[List[Dict[str,Any]], Failure]; kept the # type: ignore[override].
2. backend/worker/__main__.py ? added _serialize_and_push_outputs() helper (writes+pushes intervals.csv + report.json). _fail(rc) now writes+pushes a status="failed" report.json / intervals.csv (0 segments), best-effort, BEFORE the M6 done-marker. The success path uses the same helper. The segmenter-Failure branch already routed to _fail(3).
3. tests/test_tracknet_hybrid.py ? the two abort tests now assert isinstance(res, Failure); added a test that zero-windows stays a bare [] (not a Failure).
4. tests/test_worker.py ? new test that a segmenter Failure -> rc 3 + report.json status="failed" + done-marker returncode 3/status="failed"; strengthened the 0-segment test to assert status="success".

KEY CONSUMER CODE TO CROSS-CHECK (read the real files, do not assume):

- backend/worker/__main__.py: cmd_infer (the full control flow: validation -> segmenter -> isinstance(result, Failure) -> _fail(3); the finally: emit_indexer_report; the success tail), _write_done_marker, _write_report_json, _write_intervals_csv.
- backend/orchestration.py: cached_process_result (~line 288, requires report status=="success" to hydrate), probe_process_cache (~line 255), _read_bucket_json, _resume_one_remote_job usage.
- backend/compute/cloud_run.py: CloudRunCompute.run_infer (reads the done-marker returncode + outputs/<id>/report.json).
- backend/api/job_worker.py: _resume_one_remote_job (waits for report.json to EXIST, then reads status; marks the job failed if status != success), _run_claimed_job.
- backend/results.py: Success/Failure/Result/unwrap_or_failure contract.
- backend/main.py (isinstance Failure branch ~1557), backend/pipeline/segmenters/gemini.py & motion.py (the existing Failure precedent), yolo_hybrid.py (a PARALLEL hybrid segmenter).

CONSTRAINTS THE FIX MUST RESPECT: ruff clean, mypy clean (trajectory_hybrid is NOT quarantined; worker.__main__ IS quarantined via mypy.ini ignore_errors), full suite green, coverage floor 70%.

The full suite is already green (1736 passed) and ruff/mypy clean ? so do NOT report "tests might fail" speculatively; report only concrete defects you can point at in the code.

Only report REAL defects: a wrong/edge behavior, a silent-failure path still leaking as success, a control-plane interaction regression, an idempotency/concurrency bug, a broken test assertion, or a genuine convention violation. Cite exact file:line. If a concern is speculative or you cannot ground it in the actual code, DO NOT report it.

YOUR LENS: The fix now writes a status="failed" report.json (previously validation/unknown-seg failures wrote NO report.json at all; detector-abort wrote a false success). Cross-check every consumer for a REGRESSION or a new poisoning:

- orchestration.cached_process_result / probe_process_cache: they gate on report.status=="success".
Confirm a "failed" report never hydrates as a cache hit, and never masks a real success. Could writing a failed report at outputs/<md5>/report.json for a VALIDATION failure now block or corrupt a later legit re-submit of the same md5?
- api/job_worker._resume_one_remote_job: it waits for report.json to EXIST then reads status. Is the new "failed report exists" strictly better (no hang) or could it complete a job wrongly?
- compute/cloud_run.run_infer: reads marker returncode + report.json. With rc=3 marker + failed report, is the ComputeResult correct?
- Idempotency/concurrency: _serialize_and_push_outputs uses storage.put to the same outputs/<id>/ prefix. Any race, partial-write, or ordering issue vs the done-marker or a concurrent success? Does force=true reprocess interact badly?
- The done-marker is written AFTER the failed report push in _fail. If the report push throws (best-effort caught) but the marker is still written rc=3 ? is that the right/consistent state?

Report concrete interaction/regression defects only, with file:line and a failure scenario.

2. assistant (2026-07-06T17:26:50.828Z)

I'll start by reading the diff under review, then cross-check the consumer code.

3. user (2026-07-06T17:26:52.342Z)

```
1      diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
2      index 503f6d8..f96b495 100644
3      --- a/backend/pipeline/segmenters/trajectory_hybrid.py
4      +++ b/backend/pipeline/segmenters/trajectory_hybrid.py
5      @@ -22,7 +22,7 @@ import concurrent.futures
6      import logging
7      import os
8      import time
9      -from typing import Any, Dict, List, Optional, Tuple
10     +from typing import Any, Dict, List, Optional, Tuple, Union
11
12     import cv2
13
14     @@ -30,6 +30,9 @@ from backend.config.models import IndexingConfig
15     from backend.pipeline.detectors.base import DetectorRunner
16     from backend.pipeline.segmenters.base import BasePlaySegmenter
17     from backend.providers import provider_registry
18     +
19     +# Standardized Success|Failure contract: a detector Failure is NOT a 0-rally result.
20     +from backend.results import Failure
21     from backend.sports import sport_registry
22     from backend.utils.paths import output_base
23     from backend.utils.video import extract_video_chunk, get_video_duration
24     @@ -174,10 +177,13 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
25         video_path: str,
26         player_pool: Optional[List[str]] = None,
27         max_frames: Optional[int] = None,
```

```

28     - ) -> List[Dict[str, Any]]:
29     -     # override-ignore: the ABC declares the Result contract (Success|Failure)
30     -     # but every live segmenter still returns a bare list ? the documented
31     -     # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
32     + ) -> "Union[List[Dict[str, Any]], Failure]":
33     +     # Partial adoption of the Result contract (the ABC declares Success|Failure).
34     +     # A DETECTOR failure ? the env healthcheck not ready, or run_predict timing out
35     +     # aborting with no trajectory ? returns a ``Failure`` so it is NOT confused
with a
36     +     # genuine 0-rally video (which still returns a bare ``[]``). Every caller
37     +     # (worker.cmd_infer, main.py, orchestration) branches on ``isinstance(result,
Failure)``;
38     +     # the bare-list success/empty paths are the documented incremental gap
(CODE_MAP gotchas).
39     +     label = self.display_label
40     +     # --- Sport profile + AI provider wiring (identical across segmenters) ---
41     +     sport_name = self.config.sport
42     @@ -267,7 +273,12 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
43     +     ok, msg = runner.healthcheck()
44     +     if not ok:
45     +         logger.error("%s detector environment not ready: %s", label, msg)
46     -         return []
47     +         # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
48     +         # failed/retryable instead of writing an empty-but-"successful" result.
49     +         return Failure(
50     +             message=f"{label} detector environment not ready: {msg}",
51     +             is_recoverable=True,
52     +         )
53     +         logger.info("%s detector env OK: %s", label, msg)
54
55     +     # --- Phase 2: trajectory -> candidate rally windows ---
56     @@ -278,7 +289,17 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
57     +     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
58     +     if not csv_path:
59     +         logger.error("%s produced no trajectory; aborting.", label)
60     -         return []
61     +         # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
62     +         # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
63     +         # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
64     +         # rather than a silent 0-segment "success" (the wf294 incident).
65     +         return Failure(
66     +             message=(
67     +                 f"{label} detector produced no trajectory (timed out or aborted) ?
"
68     +                 "detector failure, not a 0-rally video."
69     +             ),
70     +             is_recoverable=True,
71     +         )
72
73     +     points = runner.parse_trajectory_csv(csv_path)
74     +     logger.info(
75     +         diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
76     +         index 461d7ca..88c50f6 100644
77     +         --- a/backend/worker/__main__.py
78     +         +++ b/backend/worker/__main__.py
79     +         @@ -114,6 +114,42 @@ def _write_report_json(
80     +             )
81
82
83     + def _serialize_and_push_outputs(

```

```

84     + scratch: str,
85     + out_uri: str,
86     + video_id: str,
87     + segments: List[dict],
88     + status: str,
89     + storage_cfg: dict,
90     + video_uri: str = "",
91     +) -> List[str]:
92     + """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
93     + ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
94     +
95     + Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
96     + ``status="failed"`` report.json in the bucket ? the control plane's source of truth
97     + (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
98     + report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
99     + EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``. """
100    + out_local = os.path.join(scratch, "outputs", video_id)
101    + os.makedirs(out_local, exist_ok=True)
102    + _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
103    + _write_report_json(
104    +     video_id,
105    +     segments,
106    +     status,
107    +     os.path.join(out_local, "report.json"),
108    +     video_uri=video_uri,
109    + )
110    + out_prefix = out_uri.rstrip("/")
111    + pushed: List[str] = []
112    + for fn in sorted(os.listdir(out_local)):
113    +     uri = f"{out_prefix}/outputs/{video_id}/{fn}"
114    +     storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
115    +     pushed.append(uri)
116    + return pushed
117    +
118    +
119    def _run_startup_checks(config: Any) -> bool:
120    """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
121    best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
122    @@ -237,9 +273,26 @@ def cmd_infer(args: argparse.Namespace) -> int:
123    )
124
125    def _fail(rc: int) -> int:
126    - # M6: on a worker-side FAILURE, write the done-marker (the REAL rc, 0 segments)
so a remote
127    - # dispatcher's bucket-poll terminates FAST + accurately instead of hanging
until its poll
128    - # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
129    + # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
130    + # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
131    + # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
132    + # truth (submit-cache probe, restart-recovery) and the alpha-user result
surface see an
133    + # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
134    + # mask the original failure. THEN the M6 done-marker (real rc + status) so a

```

```

remote
135 +      # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
136 +      try:
137 +          _serialize_and_push_outputs(
138 +              scratch,
139 +              args.out_uri,
140 +              video_id,
141 +              [],
142 +              "failed",
143 +              storage_cfg,
144 +              str(getattr(args, "video_uri", "") or ""),
145 +          )
146 +      except Exception as e: # never mask the real failure on an artifact-push
hiccup
147 +          logger.warning("[worker] could not push failure artifacts: %s", e)
148 +      # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
149 +      if getattr(args, "done_uri", None):
150 +          _write_done_marker(
151 +              args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
152 @@ -313,24 +366,16 @@ def cmd_infer(args: argparse.Namespace) -> int:
153         )
154
155         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
156 - out_local = os.path.join(scratch, "outputs", video_id)
157 - os.makedirs(out_local, exist_ok=True)
158 - _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
159 - _write_report_json(
160 + pushed = _serialize_and_push_outputs(
161 +     scratch,
162 +     args.out_uri,
163 +     video_id,
164 +     segments,
165 +     status,
166 -     os.path.join(out_local, "report.json"),
167 -     video_uri=str(getattr(args, "video_uri", "") or ""),
168 +     storage_cfg,
169 +     str(getattr(args, "video_uri", "") or ""),
170 + )
171
172 - out_prefix = args.out_uri.rstrip("/")
173 - pushed = []
174 - for fn in sorted(os.listdir(out_local)):
175 -     uri = f"{out_prefix}/outputs/{video_id}/{fn}"
176 -     storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
177 -     pushed.append(uri)
178 -
179     print(
180         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
181     )
182     diff --git a/tests/test_tracknet_hybrid.py b/tests/test_tracknet_hybrid.py
183     index 64909e7..c5ae442 100644
184     --- a/tests/test_tracknet_hybrid.py
185     +++ b/tests/test_tracknet_hybrid.py
186     @@ -5,6 +5,7 @@ healthcheck gating, candidate persistence, AI handoff, and DB
population/linking
187     from unittest.mock import MagicMock, patch
188
189     from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
190     +from backend.results import Failure
191
192
193     def _window(start=1.0, end=4.0):
194     @@ -124,7 +125,10 @@ def test_pipeline_healthcheck_failure_aborts(

```

```

195
196     seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
197     res = seg.process_video("v1", "clip.mp4")
198     assert res == []
199     + # A detector-env failure is a Failure, NOT a 0-rally [] ? so the caller marks the
run failed
200     + # instead of writing an empty "success".
201     + assert isinstance(res, Failure)
202     + assert "environment not ready" in res.message
203     + # Should bail before ever running inference.
204     mock_runner_cls.return_value.run_predict.assert_not_called()
205
206     @@ -144,7 +148,36 @@ def test_pipeline_no_trajectory_aborts(
207         mock_provider_registry.get.return_value = MagicMock()
208
209         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
210         - assert seg.process_video("v1", "clip.mp4") == []
211         + res = seg.process_video("v1", "clip.mp4")
212         + # A detector timeout/abort (run_predict yielded no trajectory) is a Failure, NOT a
0-rally [] ?
213         + # this is exactly the wf294 silent-failure that the caller must now mark
failed/retryable.
214         + assert isinstance(res, Failure)
215         + assert "no trajectory" in res.message and res.is_recoverable
216         +
217         +
218     +@_patched
219     +def test_pipeline_zero_windows_is_legit_empty_not_failure(
220         + mock_runner_cls,
221         + mock_probe,
222         + mock_dur,
223         + mock_extract,
224         + mock_provider_registry,
225         + mock_env,
226         +):
227         + """The other side of the wf294 distinction: the detector RAN fine (a trajectory was
produced +
228         + parsed) but yielded no candidate rally windows. That is a legitimately empty match
? a bare []
229         + (0-rally "success"), NOT a Failure."""
230         + mock_env.return_value = "dummy_key"
231         + mock_dur.return_value = 30.0
232         + mock_extract.return_value = True
233         + mock_runner_cls.return_value = _make_runner(windows=[]) # trajectory ok, zero
windows
234         + mock_provider_registry.get.return_value = MagicMock()
235         +
236         + seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
237         + res = seg.process_video("v1", "clip.mp4")
238         + assert res == [] and not isinstance(res, Failure)
239         + # The detector DID run (a genuine empty, not an abort).
240         + mock_runner_cls.return_value.run_predict.assert_called_once()
241
242
243     @_patched
244     diff --git a/tests/test_worker.py b/tests/test_worker.py
245     index 96edbee..63cc18e 100644
246     --- a/tests/test_worker.py
247     +++ b/tests/test_worker.py
248     @@ -10,6 +10,7 @@ import json
249     import pytest
250
251     from backend.config import load_config
252     +from backend.results import Failure
253     from backend.worker import __main__ as wmain

```

```

254
255     # ----- pure helpers
256     @@ -191,7 +192,7 @@ def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
257
258
259     class _EmptySeg:
260     -     """A segmenter that finds nothing ? the cold-start failure mode."""
261     +     """A segmenter that finds nothing ? a LEGITIMATE 0-rally match (a bare [])."""
262
263         def __init__(self, db, config):
264             pass
265     @@ -200,6 +201,20 @@ class _EmptySeg:
266         return []
267
268
269     +class _FailSeg:
270     +     """A segmenter whose DETECTOR failed/timed out ? returns a Failure (e.g. WASB
produced no
271     +     trajectory), which is NOT the same as a 0-rally match. This is the wf294 shape."""
272     +
273     +     def __init__(self, db, config):
274     +         pass
275     +
276     +     def process_video(self, video_id, path, max_frames=None):
277     +         return Failure(
278     +             message="WASB detector produced no trajectory (timed out or aborted)",
279     +             is_recoverable=True,
280     +         )
281     +
282     +
283     def test_setup_logging_levels():
284         import logging
285
286     @@ -246,12 +261,56 @@ def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path,
monkeypatch, caplog)
287         assert "0 segments" in msgs and "vidZ" in msgs
288         assert "detector_impl=native" in msgs # surfaces the resolved detector
289         assert "native runner UNAVAILABLE" in msgs # points at the likely cause
290     -     assert (
291     -         json.loads((out / "outputs" / "vidZ" / "report.json").read_text())[
292     -             "segment_count"
293     +         # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
294     +         # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as
status="failed").
295     +         report = json.loads((out / "outputs" / "vidZ" / "report.json").read_text())
296     +         assert report["segment_count"] == 0 and report["status"] == "success"
297     +
298     +
299     +def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
300     +     tmp_path, monkeypatch
301     +):
302     +     """wf294 fix: a segmenter that returns a Failure (detector timeout/abort) must NOT
be written
303     +     as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with
304     +     status="failed" so the control plane / alpha-user surface can show an error /
retry."""
305     +     monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
306     +     monkeypatch.setattr(
307     +         wmain.validator_registry,
308     +         "run_all_with_context",
309     +         lambda path, cfg: ([_OKResult()], {}),
310     +     )
311     +     monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)

```

```

312 + monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
313 + video = tmp_path / "in.mp4"
314 + video.write_bytes(b"FAKE")
315 + out = tmp_path / "bucket"
316 + done = tmp_path / "bucket" / "_dispatch" / "tok.done"
317 +
318 + rc = wmain.main(
319 +     [
320 +         "infer",
321 +         "--video-uri",
322 +         str(video),
323 +         "--out-uri",
324 +         str(out),
325 +         "--scratch",
326 +         str(tmp_path / "s"),
327 +         "--segmenter",
328 +         "wasb_hybrid",
329 +         "--done-uri",
330 +         str(done),
331 +     ]
332 -     == 0
333 )
334 + # A detector failure is rc 3 (distinct from validation's rc 2 and a clean rc 0).
335 + assert rc == 3
336 + # report.json is written as an explicit FAILURE, not a false empty "success".
337 + report = json.loads((out / "outputs" / "vidX" / "report.json").read_text())
338 + assert report["status"] == "failed" and report["segment_count"] == 0
339 + assert report["segments"] == []
340 + # ...and the completion marker the cloud dispatcher bucket-polls carries the same
verdict.
341 + marker = json.loads(done.read_text())
342 + assert marker["returncode"] == 3 and marker["status"] == "failed"
343 + assert marker["segment_count"] == 0 and marker["video_id"] == "vidX"
344
345
346     def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
347

```

4. assistant (2026-07-06T17:26:57.186Z)

I'll now read the key consumer files to cross-check the fix.

5. user (2026-07-06T17:26:59.055Z)

```

1     """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3     python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4         [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5     python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7     The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8     is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10    ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11    the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13    This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14    (validator registry ? segmenter ? report), never a forked pipeline.
15    """
16
17    from __future__ import annotations

```



```

18
19 import argparse
20 import csv
21 import json
22 import logging
23 import os
24 import re
25 import sys
26 import tempfile
27 from typing import Any, List
28
29 from backend import storage
30 from backend.compute import ComputeJobSpec, get_compute_backend
31 from backend.config import (
32     StartupCheckError,
33     enforce_startup_checks,
34     load_config,
35     probe_cuda_available,
36 )
37 from backend.infrastructure.database import Database
38 from backend.pipeline.segmenters.base import segmenter_registry
39 from backend.pipeline.validators.base import registry as validator_registry
40 from backend.reporting import emit_indexer_report
41 from backend.results import Failure
42 from backend.utils.ffmpeg import ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
43 from backend.utils.hashing import compute_video_id
44
45 logger = logging.getLogger("backend.worker")
46
47
48 def _coerce(v: str) -> Any:
49     """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
50     str."""
51     low = v.lower()
52     if low in ("true", "false"):
53         return low == "true"
54     for cast in (int, float):
55         try:
56             return cast(v)
57         except ValueError:
58             pass
59     return v
60
61 def _apply_overrides(config: Any, sets: List[str]) -> None:
62     """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
63     indexing.skip_ai_handoff=true)."""
64     for kv in sets or []:
65         key, sep, val = kv.partition("=")
66         if not sep:
67             raise ValueError(f"--set expects k=v, got {kv!r}")
68         parts = key.split(".")
69         obj = config
70         for p in parts[:-1]:
71             obj = getattr(obj, p)
72             setattr(obj, parts[-1], _coerce(val))
73
74 def _write_intervals_csv(segments: List[dict], path: str) -> None:
75     """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
76     friendly
77     artifact (same schema as the rally-annotator labels)."""
78     with open(path, "w", newline="") as f:
79         w = csv.writer(f)
80         w.writerow(["start", "end", "shot_count", "ending_reason"])

```

```

80         for s in segments:
81             w.writerow(
82                 [
83                     f"{float(s.get('start_time', 0.0)):.3f}",
84                     f"{float(s.get('end_time', 0.0)):.3f}",
85                     s.get("shot_count", ""),
86                     s.get("ending_reason", s.get("shot_count_confidence", "")),
87                 ]
88             )
89
90
91 def _write_report_json(
92     video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
93 ) -> None:
94     with open(path, "w") as f:
95         json.dump(
96             {
97                 "video_id": video_id,
98                 "status": status,
99                 "segment_count": len(segments),
100                 # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
101                 # control plane re-hydrating from outputs/<md5>/ restores the video row
102                 # with its truthful path ? without this, only the segments recover.
103                 "video_uri": video_uri,
104                 "segments": [
105                     {
106                         "start": float(s.get("start_time", 0.0)),
107                         "end": float(s.get("end_time", 0.0)),
108                     }
109                     for s in segments
110                 ],
111             },
112             f,
113             indent=2,
114         )
115
116
117 def _serialize_and_push_outputs(
118     scratch: str,
119     out_uri: str,
120     video_id: str,
121     segments: List[dict],
122     status: str,
123     storage_cfg: dict,
124     video_uri: str = "",
125 ) -> List[str]:
126     """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
127     push both to
128     ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
129
130     Shared by the SUCCESS path and the failure path (``_fail``) so a detector
131     timeout/abort leaves a
132     ``status="failed"`` report.json in the bucket ? the control plane's source of truth
133     (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
134     non-``success``
135     report as "no usable result" ? retry, and the restart-recovery poll waits for
136     report.json to
137     EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``.
138     """
139     out_local = os.path.join(scratch, "outputs", video_id)
140     os.makedirs(out_local, exist_ok=True)
141     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
142     _write_report_json(
143         video_id,
144         segments,
145         status,

```

```

141         os.path.join(out_local, "report.json"),
142         video_uri=video_uri,
143     )
144     out_prefix = out_uri.rstrip("/")
145     pushed: List[str] = []
146     for fn in sorted(os.listdir(out_local)):
147         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
148         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
149         pushed.append(uri)
150     return pushed
151
152
153 def _run_startup_checks(config: Any) -> bool:
154     """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
155     best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
156     cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
157     (returns True, ZERO work) when no deployment.profile is selected.
158
159     The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
160     torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
161     WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
162     no-op of work, not just of output."""
163     profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164     cuda = (
165         probe_cuda_available() if profile else None
166     ) # torch-free on the default-OFF path
167     try:
168         enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169         return True
170     except StartupCheckError:
171         return False # already logged at ERROR by enforce_startup_checks
172
173
174 def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175     """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
176     outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
177     container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
178
179     A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
180     poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
181     timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
182     from backend.infrastructure.execution_policy import PolicyTimeout
183
184     spec = ComputeJobSpec(
185         video_uri=args.video_uri,
186         out_uri=args.out_uri,
187         segmenter=args.segmenter,
188         config_overrides=tuple(args.set or ()),
189     )
190     try:
191         result = compute.run_infer(spec)
192     except PolicyTimeout as e:
193         logger.warning(

```

```

194         "[worker] remote compute did not complete (no marker within budget): %s", e
195     )
196     return 4
197     logger.info(
198         "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
199         result.video_id,
200         result.returncode,
201         result.outputs_uri,
202     )
203     return result.returncode
204
205
206     def _write_done_marker(
207         done_uri: str,
208         video_id: str,
209         returncode: int,
210         segment_count: int,
211         status: str,
212         storage_cfg: dict,
213         scratch: str,
214     ) -> None:
215         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
216         for a remote
217         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
218         return."""
219         try:
220             marker = {
221                 "video_id": video_id,
222                 "returncode": returncode,
223                 "segment_count": segment_count,
224                 "status": status,
225             }
226             local = os.path.join(scratch, "_done.json")
227             with open(local, "w", encoding="utf-8") as f:
228                 json.dump(marker, f)
229             storage.put(local, done_uri, config=storage_cfg)
230             logger.info("[worker] wrote completion marker -> %s", done_uri)
231         except Exception as e: # never fail a good run because the marker push hiccuped
232             logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
233
234     def cmd_infer(args: argparse.Namespace) -> int:
235         config = load_config(args.config) if args.config else load_config()
236         _apply_overrides(config, args.set)
237         if not _run_startup_checks(config):
238             return 2
239
240         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
241         # process.
242         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
243         # through to the
244         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
245         # (the
246         # dispatcher forces compute_target=local_gpu), so it never recursively re-
247         # dispatches.
248         compute = get_compute_backend(config)
249         if compute is not None:
250             return _dispatch_remote(compute, args)
251
252         storage_cfg = config.storage.model_dump()
253
254         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
255         os.makedirs(scratch, exist_ok=True)
256         config.output.output_dir = scratch

```

```

253         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
254         local_video = storage.fetch(
255             args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
256         )
257         print(f"[worker] pulled {args.video_uri} -> {local_video}")
258
259         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
260         video_id = compute_video_id(local_video)
261
262         # 3. VALIDATE (same registry as the main CLI).
263         val_results, val_context = validator_registry.run_all_with_context(
264             local_video, config
265         )
266         failures = [r for r in val_results if not r.passed]
267         db = Database(os.path.join(scratch, config.output.db_name))
268         db.add_video(
269             video_id,
270             local_video,
271             "failed" if failures else "processed",
272             [r.model_dump() for r in val_results],
273         )
274
275         def _fail(rc: int) -> int:
276             # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
277             # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
278             # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
279             # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
280             # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
281             # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
282             # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
283             try:
284                 _serialize_and_push_outputs(
285                     scratch,
286                     args.out_uri,
287                     video_id,
288                     [],
289                     "failed",
290                     storage_cfg,
291                     str(getattr(args, "video_uri", "") or ""),
292                 )
293             except Exception as e: # never mask the real failure on an artifact-push hiccup
294                 logger.warning("[worker] could not push failure artifacts: %s", e)
295             # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
296             if getattr(args, "done_uri", None):
297                 _write_done_marker(
298                     args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
299                 )
300             return rc
301
302         segments: List[dict] = []
303         segmenter_actual = None
304         segmentation_ran = False
305         status = "failed"
306         try:
307             if failures:
308                 print(

```

```

309         "[worker] validation FAILED: "
310         + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
311     )
312     return _fail(2)
313     seg_name = args.segmenter or config.indexing.default_segmenter
314     segmenter_cls = segmenter_registry.get(seg_name)
315     if not segmenter_cls:
316         print(
317             f"[worker] unknown segmenter: {seg_name!r} "
318             f"(have {list(segmenter_registry.list_available())})"
319         )
320         return _fail(2)
321     segmenter_actual = seg_name
322     result = segmenter_cls(db, config).process_video(
323         video_id, local_video, max_frames=args.max_frames
324     )
325     segmentation_ran = True
326     if isinstance(result, Failure):
327         print(f"[worker] segmenter FAILED: {result.message}")
328         return _fail(3)
329     segments = result.value if hasattr(result, "value") else result
330     status = "success"
331     # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
332     # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
333     # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
334     if not segments:
335         detector_impl = (
336             os.environ.get("RALLY_DETECTOR_IMPL")
337             or getattr(config.indexing, "detector_impl", None)
338             or "auto"
339         )
340         logger.warning(
341             "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
342             "The detector substrate yielded no usable trajectories/windows ? check
the detector "
343             "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
344             "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
345             "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
346             "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
347             "detections, or the input resolution differs from the tuned proxy
resolution. "
348             "Re-run with -v for the native command + frame counts.",
349             video_id,
350             segmenter_actual,
351             detector_impl,
352         )
353     finally:
354         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
355         emit_indexer_report(
356             db=db,
357             video_id=video_id,
358             video_path=local_video,
359             config=config,
360             val_results=val_results,
361             val_context=val_context,
362             segmenter_requested=args.segmenter,
363             segmenter_actual=segmenter_actual,

```

```

364         was_reingest=False,
365         segmentation_ran=segmentation_ran,
366     )
367
368     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
369     pushed = _serialize_and_push_outputs(
370         scratch,
371         args.out_uri,
372         video_id,
373         segments,
374         status,
375         storage_cfg,
376         str(getattr(args, "video_uri", "") or ""),
377     )
378
379     print(
380         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}"
381     )
382     for u in pushed:
383         print("    ", u)
384
385     # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
386     # its bucket-poll
387     # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
388     FAILURES (rc 2/3)
389     # write their own marker via _fail() above, so the dispatcher fails FAST either way
390     (no 30-min hang
391     # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
392     local run).
393     if getattr(args, "done_uri", None):
394         _write_done_marker(
395             args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
396         )
397     return 0
398
399     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
400     so a
401     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
402     collision).
403     _VIDEO_EXTS = ("mp4", "mov", "mkv", "avi", "webm", "m4v")
404
405     #: Leading ISO-date prefix on a canonical label name (`labels/<YYYY-MM-
406     DD>_<name>.rallies.csv`,
407     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
408     but is NOT
409     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
410     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
411
412     def _label_clip_name(fname: str) -> str:
413         """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
414         filename.
415
416         Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
417         prefix, so a
418         canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
419
420         * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094`` (canonical, date-
421         stamped)
422         * ``GX020094.rallies.csv`` -> ``GX020094`` (legacy bare ? same
423         stem)
424         * ``GX020094_proxy.rallies.csv`` -> ``GX020094_proxy`` (``_proxy`` is part of
425         the name, kept)

```

```

415         * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)
416
417         Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
418         video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS $7b #1).
419         ""
420         name = fname.replace(".rallies.csv", "").replace(".csv", "")
421         return _LABEL_DATE_PREFIX.sub("", name)
422
423
424     def _probe_video_fps_width(path: str):
425         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
426
427         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
428         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
429         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
430         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
431         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
432         rather than guesses.
433         ""
434         import json
435         import subprocess
436
437         try:
438             out = subprocess.check_output(
439                 [
440                     ffprobe_bin(),
441                     "-v",
442                     "error",
443                     "-select_streams",
444                     "v:0",
445                     "-show_entries",
446                     "stream=r_frame_rate,width",
447                     "-of",
448                     "json",
449                     path,
450                 ],
451                 stderr=subprocess.DEVNULL,
452             ).decode()
453             st = (json.loads(out).get("streams") or [{}])[0]
454             num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
455             fps = float(num) / float(den or "1")
456             width = float(st.get("width") or 0)
457             if fps > 0 and width > 0:
458                 return fps, width
459         except (
460             subprocess.SubprocessError,
461             ValueError,
462             KeyError,
463             OSError,
464             json.JSONDecodeError,
465         ):
466             pass
467         return None
468
469
470     def _resolve_train_detector(args: argparse.Namespace, config: Any):
471         """Build the trajectory DetectorRunner for ``--trajectory-source detector``, honouring

```



```

472 detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
473 stub=CPU mock). Returns the runner, or prints an error + returns None.
474
475 Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
476 detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
477 no shuttle detector and is rejected.
478 """
479 from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
480
481 seg_cls = segmenter_registry.get(args.segmenter)
482 if not seg_cls:
483     print(
484         f"[worker] unknown segmenter: {args.segmenter!r} "
485         f"(have {list(segmenter_registry.list_available())})"
486     )
487     return None
488 seg = seg_cls(None, config)
489 if not isinstance(seg, TrajectoryHybridSegmenter):
490     print(
491         f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
492         f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid)."
493     )
494     return None
495 try:
496     runner, _ = seg._resolve_runner(config.indexing)
497 except NotImplementedError as e:
498     # e.g. detector_impl='native' for a family without a native runner (tracknet, or
499     # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
500     print(f"[worker] detector not available: {e}")
501     return None
502 ok, msg = runner.healthcheck()
503 if not ok:
504     print(f"[worker] detector environment not ready: {msg}")
505     return None
506 print(f"[worker] detector ready ({args.segmenter}): {msg}")
507 return runner
508
509
510 def cmd_train(args: argparse.Namespace) -> int:
511     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
512     to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
513
514     Two trajectory sources (the train pipeline + harness are identical either way):
515     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
516         shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
517     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
518         DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
519         is the M3.5 "first real training" path; only the trajectory source flips.
520     """
521     import subprocess
522
523     config = load_config(args.config) if args.config else load_config()
524     _apply_overrides(config, args.set)
525     if not _run_startup_checks(config):
526         return 2
527     storage_cfg = config.storage.model_dump()

```

```

528
529     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
530     labels_dir = os.path.join(scratch, "labels")
531     traj_dir = os.path.join(scratch, "trajectories")
532     videos_dir = os.path.join(scratch, "videos")
533     os.makedirs(labels_dir, exist_ok=True)
534     os.makedirs(traj_dir, exist_ok=True)
535
536     corpus = args.corpus_uri.rstrip("/")
537     label_uris = sorted(
538         u
539         for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
540         if u.lower().endswith(".csv")
541     )
542     if len(label_uris) < 2:
543         print(
544             f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
545             f"under {corpus}/labels/"
546         )
547         return 2
548
549     # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
550     runner = None
551     video_by_stem: dict = {}
552     if args.trajectory_source == "detector":
553         os.makedirs(videos_dir, exist_ok=True)
554         runner = _resolve_train_detector(args, config)
555         if runner is None:
556             return 2
557         for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
558             vname = storage.parse_uri(vuri).name
559             if not vname.lower().endswith(_VIDEO_EXTS):
560                 continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
561             video_by_stem[os.path.splitext(vname)[0]] = vuri
562
563     print(f"[worker] trajectory source: {args.trajectory_source}")
564     specs: List[dict] = []
565     for uri in label_uris:
566         fname = storage.parse_uri(uri).name # e.g. 2026-06-22_GX020094.rallies.csv
567         name = _label_clip_name(fname) # -> GX020094 (date prefix stripped)
568         local_label = storage.fetch(
569             uri, os.path.join(labels_dir, fname), config=storage_cfg
570         )
571         if args.trajectory_source == "detector":
572             vuri = video_by_stem.get(name)
573             if not vuri:
574                 print(
575                     f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
576                 )
577                 continue
578             local_video = storage.fetch(
579                 vuri,
580                 os.path.join(videos_dir, storage.parse_uri(vuri).name),
581                 config=storage_cfg,
582             )
583             video_id = compute_video_id(local_video)
584             # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
585             # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
586             meta = _probe_video_fps_width(local_video)
587             if meta is None:

```

```

588         print(
589             f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
590         )
591         continue
592     fps, frame_width = meta
593     traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
594     if not traj:
595         print(
596             f"[worker] detector produced no trajectory for {name!r} ? skipping.")
597         )
598         continue
599     specs.append(
600         {
601             "name": name,
602             "trajectory": traj,
603             "labels": local_label,
604             "fps": fps,
605             "frame_width": frame_width,
606         }
607     )
608     else:
609         from backend.pipeline.detectors.stub_runner import (
610             synth_trajectory_from_labels,
611         )
612
613         traj = os.path.join(traj_dir, f"{name}_traj.csv")
614         synth_trajectory_from_labels(
615             local_label, traj, fps=args.fps, frame_width=args.frame_width
616         )
617         specs.append(
618             {
619                 "name": name,
620                 "trajectory": traj,
621                 "labels": local_label,
622                 "fps": args.fps,
623                 "frame_width": args.frame_width,
624             }
625         )
626
627     if len(specs) < 2:
628         print(
629             f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}")."
630         )
631         return 2
632     manifest_path = os.path.join(scratch, "manifest.json")
633     with open(manifest_path, "w", encoding="utf-8") as f:
634         json.dump(specs, f, indent=2)
635     src_label = (
636         "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
637     )
638     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
639
640     out_dir = os.path.join(scratch, "artifacts")
641     cmd = [
642         sys.executable,
643         "-m",
644         "training.gen0.harness",
645         "--manifest",
646         manifest_path,
647         "--out",
648         out_dir,
649         "--version",
650         args.version,

```

```

651     ]
652     if args.floor:
653         floor_local = (
654             storage.fetch(
655                 args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
656             )
657             if "://" in args.floor
658             else args.floor
659         )
660         cmd += ["--floor", floor_local]
661     print("[worker] training:", " ".join(cmd))
662     rc = subprocess.run(cmd).returncode
663     if rc != 0:
664         print(f"[worker] gen0 harness failed (rc={rc})")
665         return rc
666
667     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
668     bundle = os.path.join(out_dir, args.version)
669     out_prefix = args.out_uri.rstrip("/")
670     pushed = []
671     for fn in sorted(os.listdir(bundle)):
672         uri = f"{out_prefix}/weights/{args.version}/{fn}"
673         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
674         pushed.append(uri)
675     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
676     for u in pushed:
677         print("    ", u)
678     return 0
679
680
681 def build_parser() -> argparse.ArgumentParser:
682     p = argparse.ArgumentParser(
683         prog="python -m backend.worker",
684         description="Storage-aware worker: pull ? run pipeline ? push.",
685     )
686     p.add_argument(
687         "-v",
688         "--verbose",
689         action="store_true",
690         help="DEBUG logging (the native detector command, frame counts, per-stage
691         detail)",
692     )
693     sub = p.add_subparsers(dest="cmd", required=True)
694
695     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
696     pi.add_argument(
697         "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
698     )
699     pi.add_argument(
700         "--out-uri",
701         required=True,
702         help="output prefix (outputs/<video_id>/? is appended)",
703     )
704     pi.add_argument(
705         "--segmenter", default=None, help="default: indexing.default_segmenter"
706     )
707     pi.add_argument(
708         "--config", default=None, help="config.json path (default: ./config.json)"
709     )
710     pi.add_argument(
711         "--scratch", default=None, help="local scratch dir (default: a temp dir)"
712     )
713     pi.add_argument("--max-frames", type=int, default=None)
714     pi.add_argument(
715         "--done-uri",

```

```

715         default=None,
716         help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
717         "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
718         "Default-OFF: omit it for the normal local run.",
719     )
720     pi.add_argument(
721         "--set",
722         action="append",
723         default=[],
724         metavar="k=v",
725         help="config override, e.g. --set indexing.skip_ai_handoff=true",
726     )
727     pi.set_defaults(func=cmd_infer)
728
729     pt = sub.add_parser(
730         "train",
731         help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
732     )
733     pt.add_argument(
734         "--corpus-uri",
735         required=True,
736         help="prefix containing labels/ (+ videos/ for "
737         "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
738     )
739     pt.add_argument(
740         "--out-uri",
741         required=True,
742         help="output prefix (weights/<version>/ is appended)",
743     )
744     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
745     pt.add_argument(
746         "--trajectory-source",
747         choices=["synth", "detector"],
748         default="synth",
749         help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
750         "detector = REAL shuttle trajectories via the resolved DetectorRunner "
751         "(needs videos/ in the corpus; honours detector_impl).",
752     )
753     pt.add_argument(
754         "--segmenter",
755         default="wasb_hybrid",
756         help="trajectory hybrid whose detector is used for --trajectory-source "
757         "detector",
758     )
759     pt.add_argument(
760         "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
761     )
762     pt.add_argument("--config", default=None)
763     pt.add_argument("--scratch", default=None)
764     pt.add_argument(
765         "--fps",
766         type=float,
767         default=30.0,
768         help="synth trajectory fps (detector probes the video)",
769     )
770     pt.add_argument(
771         "--frame-width",
772         type=float,
773         default=1280.0,
774         help="synth trajectory frame width (detector probes the video)",
775     )
776     pt.add_argument("--set", action="append", default=[], metavar="k=v")
777     pt.set_defaults(func=cmd_train)
778     return p

```

```

779
780     def _setup_logging(verbose: bool) -> None:
781         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
782         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
783         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
784         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
785         entry), not at import, so importing the module never reconfigures a caller's
logging."""
786         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
787         # configured logging at import), which would leave the level at WARNING and re-
suppress the
788         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
789         logging.basicConfig(
790             level=logging.DEBUG if verbose else logging.INFO,
791             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
792             datefmt="%H:%M:%S",
793             force=True,
794         )
795
796
797     def main(argv=None) -> int:
798         args = build_parser().parse_args(argv)
799         _setup_logging(getattr(args, "verbose", False))
800         return args.func(args)
801
802
803     if __name__ == "__main__":
804         sys.exit(main())
805

```

6. user (2026-07-06T17:26:59.070Z)

```

1     """Lightweight Result type for standardizing success vs. failure across the pipeline.
2
3     This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5     The goal is to make it impossible to confuse "no data / no rallies" with "something
actually failed"
6     (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
7
8     We use simple dataclasses so callers can use isinstance checks or structural pattern
matching.
9     This is intentionally small and incremental ? we start by applying it to the segmenter
10    and provider layers (the biggest sources of silent [] returns today).
11
12    The types are generic where it makes sense. Callers can still access .value on Success
13    or .message on Failure.
14
15    This module has no dependencies on the rest of the app.
16    """
17
18    from __future__ import annotations
19
20    from dataclasses import dataclass, field
21    from typing import Any, Generic, List, Optional, Tuple, TypeVar, Union
22
23    T = TypeVar("T")
24
25

```

```

26 @dataclass
27 class Success(Generic[T]):
28     """Represents a successful outcome with an optional list of non-fatal warnings."""
29
30     value: T
31     warnings: List[str] = field(default_factory=list)
32
33     def is_success(self) -> bool:
34         return True
35
36     def unwrap(self) -> T:
37         """Return the value. Only safe to call if you already know it's a Success."""
38         return self.value
39
40
41 @dataclass
42 class Failure:
43     """Represents a failure with a human-readable message and optional details."""
44
45     message: str
46     details: Optional[dict[str, Any]] = None
47     is_recoverable: bool = False # e.g. missing API key (recoverable) vs internal crash
48
49     def is_success(self) -> bool:
50         return False
51
52     def unwrap(self) -> Any:
53         """Raise to make misuse obvious in code that assumes success."""
54         raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
55
56
57 # Convenient alias for use in type hints: Result[list[dict], Failure]
58 # Result[ValueType] means Success[ValueType] | Failure
59 # (the Failure "error" type is fixed for this contract)
60 Result = Union[Success[T], Failure]
61
62
63 def unwrap_or_failure(result: Any) -> Tuple[Optional[Any], Optional[Failure]]:
64     """Split a segmenter/provider return into ``(value, failure)`` so a caller branches
once.
65
66     - ``Failure`` -> ``(None, failure)``
67     - ``Success(value=v)`` -> ``(v, None)``
68     - a bare value (legacy code that predates this contract and still returns a raw
list)
69     -> ``(value, None)``
70
71     This is the same isinstance/``.value`` unwrap that ``backend/main.py`` does inline;
callers
72     that forget it (e.g. ``len(process_video(...))``) crash with ``TypeError: ... has no
len()``
73     on a ``Success`` dataclass. ``result`` is typed ``Any`` because the contract is
heterogeneous
74     (Success | Failure | legacy bare value)."""
75     if isinstance(result, Failure):
76         return None, result
77     value = result.value if hasattr(result, "value") else result
78     return value, None
79

```

7. assistant (2026-07-06T17:27:05.195Z)

Now let me read the orchestration, cloud_run, and job_worker consumers.

8. user (2026-07-06T17:27:06.600Z)

```

220         "Pass force=true to reprocess this video."
221     ),
222     "results": validation_results or [],
223     "play_segments": segments,
224     "cached": True,
225     "compute": {"path": "cached", "requested": requested_compute},
226 }
227
228
229 def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
230     """Fetch + parse one small JSON object (cloud backends require a real dst ? the
231     ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
232     decide."""
233     ref = storage.parse_uri(uri)
234     with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
235         local = storage.fetch(
236             uri, os.path.join(td, ref.name or "report.json"), config=storage_config
237         )
238         with open(local, encoding="utf-8") as f:
239             loaded = json.load(f)
240         return loaded if isinstance(loaded, dict) else None
241
242     # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
243     hits
244     # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
245     # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
246     instance
247     # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
248     _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
249     _REHYDRATE_LOCKS_GUARD = threading.Lock()
250
251     def _rehydrate_lock(video_id: str) -> threading.Lock:
252         with _REHYDRATE_LOCKS_GUARD:
253             return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
254
255     def probe_process_cache(db, cfg, video_id: str) -> bool:
256         """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
257         bucket)?
258         Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
259         1
260         ordering). Used to decide whether a submit can skip the local-scratch fetch guard
261         (blocker 2): a True probe means the request should never need CP scratch. Best-
262         effort:
263         errors report False (? the normal dispatch path with its guards)."""
264         try:
265             existing = db.get_video(video_id)
266             if (
267                 existing
268                 and existing.get("status") == "processed"
269                 and db.query_play_segments(video_id, {})
270             ):
271                 return True
272             sc = _storage_settings(cfg)
273             if not sc or (sc.get("backend") or "local") == "local":
274                 return False
275             output_root = (sc.get("output_root") or "").rstrip("/")
276             if not output_root:
277                 return False
278             report_uri = f"{output_root}/outputs/{video_id}/report.json"
279             if not storage.exists(report_uri, config=sc):
280                 return False

```



```

279         report = _read_bucket_json(report_uri, sc)
280         return bool(report and report.get("status") == "success")
281     except Exception: # noqa: BLE001 ? a failed probe degrades to the guarded dispatch
path
282         logger.warning(
283             "[API] cache probe failed for %s ? treating as a miss", video_id,
exc_info=True
284         )
285         return False
286
287
288     def cached_process_result(
289         db, cfg, video_id: str, video_path: str, requested_compute=None
290     ) -> Optional[Dict[str, Any]]:
291         """A ready-to-serve /api/process result when this video's detection ALREADY exists ?
292         the DB first, then the bucket (``outputs/<md5>/report.json`` with ``status=success``
293         re-ingests via ``_ingest_remote_segments``): **the DB is a cache, the bucket is the
294         source of truth** (#484 instant re-runs; the #485 re-hydrate mechanism). ``None`` ?
no
295         completed work found (or the check failed) ? the caller dispatches normally. The
bucket
296         branch mirrors the real cloud ingest exactly (watermarks + ``_finalize_reingest``),
so a
297         partial earlier ingest is replaced, never double-counted.
298
299         Concurrency (#492 review blocker 1): the whole check-then-ingest runs under a
300         per-``video_id`` lock ? a concurrent caller blocks, then finds the winner's rows via
the
301         in-lock DB re-check and reuses them (one segment set, ever). Callers must hold a
CLAIMED
302         job before invoking (writes happen post-claim)."""
303         try:
304             with _rehydrate_lock(video_id):
305                 existing = db.get_video(video_id)
306                 if existing and existing.get("status") == "processed":
307                     segs = db.query_play_segments(video_id, {})
308                     if segs:
309                         return _cached_process_payload(
310                             video_id,
311                             existing.get("validation_results", []),
312                             segs,
313                             requested_compute,
314                         )
315                 sc = _storage_settings(cfg)
316                 if not sc or (sc.get("backend") or "local") == "local":
317                     return None
318                 output_root = (sc.get("output_root") or "").rstrip("/")
319                 if not output_root:
320                     return None
321                 outputs_uri = f"{output_root}/outputs/{video_id}"
322                 report_uri = f"{outputs_uri}/report.json"
323                 if not storage.exists(report_uri, config=sc):
324                     return None
325                 report = _read_bucket_json(report_uri, sc)
326                 if not report or report.get("status") != "success":
327                     return None
328                 if not video_path:
329                     # Read-path re-hydrate (#485): a wiped DB has no source record. The
report
330                     # carries the worker's --video-uri (P3); older reports fall back to the
331                     # staged-canonical videos/<md5>/ prefix; else "" ? the segments still
332                     # recover, and the next same-md5 submit re-binds the truthful path via
333                     # add_video's upsert.
334                     video_path = str(report.get("video_uri") or "") or _first_object_under(
335                         f"{output_root}/videos/{video_id}/", sc

```

```

336         )
337         play_wm, cand_wm = db.segment_watermarks(video_id)
338         db.add_video(video_id, video_path, "processed", [])
339         n = _ingest_remote_segments(db, video_id, outputs_uri, sc)
340         if n <= 0:
341             return None
342         segments = [
343             s
344             for s in db.query_play_segments(video_id, {})
345             if int(s.get("id", 0)) > play_wm
346         ]
347         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
348         if not segments:
349             return None
350         logger.info(
351             "[API] re-hydrated %d segments for video_id=%s from %s (submit-time
cache hit)",
352             n,
353             video_id,
354             outputs_uri,
355         )
356         return _cached_process_payload(video_id, [], segments, requested_compute)
357     except Exception: # noqa: BLE001 ? cache misses must degrade to a normal dispatch
358         logger.warning(
359             "[API] submit-time cache check failed for %s ? dispatching normally",
360             video_id,
361             exc_info=True,
362         )
363         return None
364
365
366     def _first_object_under(prefix_uri: str, storage_config: dict) -> str:
367         """The first object URI under a bucket prefix, or "" ? best-effort source
recovery."""
368         try:
369             return next(iter(storage.list_uris(prefix_uri, config=storage_config)), "")
370         except Exception: # noqa: BLE001 ? recovery nicety, never a failure
371             return ""
372
373
374     _MD5_HEX_RE = re.compile(r"[0-9a-f]{32}")
375
376
377     def read_path_rehydrate(db, cfg, video_id: str) -> bool:
378         """#485, the read half of "DB = cache, bucket = truth": before a rally READ reports
379         nothing for an md5-shaped id, try re-hydrating the video from ``outputs/<md5>/`` ?
380         so

```

9. user (2026-07-06T17:27:06.986Z)

```

1         """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2         for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4         Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6             --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8             completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9             it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the

```

```

10         cold-start is absorbed by the async poll. On a poll TIMEOUT (`max_wait_sec` can
undercut the
11         job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-
but-healthy
12         worker that finished just past the deadline is rescued, not wasted ? then best-
effort CANCELS
13         the still-running execution so the L4 doesn't keep billing until ``--task-
timeout``.
14         3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
15         ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
16
17         The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
18         so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
19
20         The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
21         with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
22         owner-verified on the M7 real run.
23         ""
24
25         from __future__ import annotations
26
27         import json
28         import logging
29         import os
30         import tempfile
31         import uuid
32         from abc import ABC, abstractmethod
33         from typing import Any, Callable, List, Optional
34
35         from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
36         from backend.infrastructure.execution_policy import (
37             DEFAULT_POLICY,
38             ExecutionPolicy,
39             PolicyTimeout,
40             poll_until,
41         )
42         from backend.storage import fetch, get_storage, parse_uri
43
44         LOG = logging.getLogger("compute.cloud_run")
45
46         # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
47         # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
48         _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
49             "deployment.compute_target=local_gpu",
50             "indexing.detector_impl=native",
51         )
52
53
54         class CloudRunJobClient(ABC):
55             """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
56
57             Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
58             overrides the job's container ``args`` for this execution and starts it."""
59
60             @abstractmethod
61             def run_job(
62                 self,

```

```

63         job_name: str,
64         argv: List[str],
65         *,
66         region: str,
67         project: Optional[str] = None,
68     ) -> str:
69         """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
70         execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
71
72     @abstractmethod
73     def cancel_execution(self, execution: str) -> None:
74         """Cancel a running execution by the name ``run_job`` returned. Called when the
bucket-poll
75         times out so the abandoned execution doesn't keep billing the GPU until
``--task-timeout``.
76         May raise ? the caller treats every failure as best-effort (the original poll
timeout is
77         NEVER masked by a cancel error)."""
78
79
80     class GoogleCloudRunJobClient(CloudRunJobClient):
81         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
82         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
83
84         def run_job(
85             self,
86             job_name: str,
87             argv: List[str],
88             *,
89             region: str,
90             project: Optional[str] = None,
91         ) -> str:
92             # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
93             # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
94             # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
95             if not project:
96                 raise RuntimeError(
97                     "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
98                     "to resolve the job path; set it on the control plane (see
get_compute_backend)."
99                 )
100             try:
101                 from google.cloud import run_v2 # type: ignore[attr-defined]
102             except Exception as exc: # pragma: no cover - real SDK not present in CI
103                 raise RuntimeError(
104                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
105                     "control plane (it is intentionally NOT a CI/test dependency)."
106                 ) from exc
107             client = (
108                 run_v2.JobsClient()
109             ) # pragma: no cover - exercised only on the real M7 run
110             name = client.job_path(project, region, job_name)
111             overrides = run_v2.RunJobRequest.Overrides(
112                 container_overrides=[
113                     run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
114                 ]

```

```

115         )
116         op = client.run_job(
117             request=run_v2.RunJobRequest(name=name, overrides=overrides)
118         )
119         return (
120             getattr(op, "metadata", None)
121             and getattr(op.metadata, "name", "")
122             or "execution"
123         )
124
125     def cancel_execution(self, execution: str) -> None:
126         # run_job falls back to the literal string "execution" when the operation
127         # metadata carries
128         # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
129         import
130         # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
131         (where
132             # google-cloud-run is absent).
133             if "/executions/" not in (execution or ""):
134                 raise RuntimeError(
135                     f"not a fully-qualified Cloud Run execution name: {execution!r} ?
136                     expected "
137                     "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
138                     resolve "
139                     "one from the operation metadata, so there is nothing to cancel by
140                     name)."
141                 )
142             try:
143                 from google.cloud import run_v2 # type: ignore[attr-defined]
144             except Exception as exc: # pragma: no cover - real SDK not present in CI
145                 raise RuntimeError(
146                     "google-cloud-run is required to cancel a Cloud Run execution; install
147                     it on the "
148                     "control plane (it is intentionally NOT a CI/test dependency)."
149                 ) from exc
150             client = (
151                 run_v2.ExecutionsClient()
152             ) # pragma: no cover - exercised only on a real run
153             client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))
154
155 class CloudRunCompute(ComputeBackend):
156     """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
157
158     def __init__(
159         self,
160         *,
161         run_client: CloudRunJobClient,
162         job_name: str,
163         region: str,
164         project: Optional[str] = None,
165         storage_config: Optional[dict] = None,
166         policy: ExecutionPolicy = DEFAULT_POLICY,
167         token: Optional[str] = None,
168         container_overrides: Optional[tuple[str, ...]] = None,
169     ) -> None:
170         self._client = run_client
171         self._job_name = job_name
172         self._region = region
173         self._project = project
174         self._storage_config = storage_config or {}
175         self._policy = policy
176         # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
177         collide.
178         # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic

```

```

(tests).
172         self._token = token
173         self._container_overrides = (
174             _DEFAULT_CONTAINER_OVERRIDES
175             if container_overrides is None
176             else tuple(container_overrides)
177         )
178
179     def run_infer(
180         self,
181         spec: ComputeJobSpec,
182         *,
183         on_wait: "Callable[[float], None] | None" = None,
184     ) -> ComputeResult:
185         out_root = spec.out_uri.rstrip("/")
186         token = self._token or uuid.uuid4().hex
187         done_uri = f"{out_root}/_dispatch/{token}.done"
188         argv: List[str] = [
189             "infer",
190             "--video-uri",
191             spec.video_uri,
192             "--out-uri",
193             spec.out_uri,
194             "--done-uri",
195             done_uri,
196         ]
197         if spec.segmenter:
198             argv += ["--segmenter", spec.segmenter]
199         for kv in (*spec.config_overrides, *self._container_overrides):
200             argv += ["--set", kv]
201
202         execution = self._client.run_job(
203             self._job_name, argv, region=self._region, project=self._project
204         )
205         LOG.info(
206             "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
207             self._job_name,
208             execution,
209             done_uri,
210         )
211
212         try:
213             marker = poll_until(
214                 lambda: self._read_marker(done_uri),
215                 self._policy,
216                 label="cloud-run-infer",
217                 logger=LOG,
218                 # Live heartbeat (#482): each still-waiting tick reaches the caller so
219                 # job.progress moves during the whole GPU run instead of freezing.
220                 on_tick=on_wait,
221             )
222         except PolicyTimeout as timeout_exc:
223             marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
224             video_id = str(marker.get("video_id", ""))
225             # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
226             # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
227             rc = int(marker.get("returncode", 1))
228             if not video_id:
229                 LOG.warning(
230                     "cloud-run: job=%s completion marker present but empty/unreadable (no "
231                     "video_id) ? treating as failure",
232                     self._job_name,
233                 )

```

```

234         rc = rc or 1
235         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
236         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
237         LOG.info(
238             "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
239         )
240         return ComputeResult(
241             returncode=rc,
242             outputs_uri=outputs_uri,
243             video_id=video_id,
244             report=report,
245             execution=str(execution or ""),
246         )
247
248     def _handle_poll_timeout(
249         self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
250     ) -> dict:
251         """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.
252
253         The poll budget (`max_wait_sec`, default 1800s) can undercut the deployed
254         job's own ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
255         the deadline:
256         ONE final marker check rescues that completed GPU run instead of wasting it. If
257         the marker
258         is genuinely absent, the execution is still running ? best-effort cancel it
259         (otherwise the
260         L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
261         carrying the
262         execution name so the operator can verify the state in the GCP console. A cancel
263         failure
264         NEVER masks the original timeout."""
265         try:
266             late = self._read_marker(done_uri)
267         except Exception: # noqa: BLE001 ? the final check is best-effort too: absence
268             ? failure
269             LOG.warning(
270                 "cloud-run: final post-timeout marker check failed ? treating as
271                 absent",
272                 exc_info=True,
273             )
274             late = None
275         if late is not None:
276             LOG.warning(
277                 "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
278                 check "
279                 "? rescuing the completed result",
280                 self._job_name,
281                 execution,
282             )
283             return late
284         try:
285             self._client.cancel_execution(execution)
286         except Exception as cancel_exc: # noqa: BLE001 ? best-effort: never mask the
287             timeout
288             cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
289             LOG.warning(
290                 "cloud-run: best-effort cancel of execution=%s failed: %s",
291                 execution,
292                 cancel_exc,
293                 exc_info=True,
294             )
295         else:
296             cancel_note = "cancel requested"
297             LOG.info("cloud-run: requested cancel of execution=%s", execution)

```

```

289         raise PolicyTimeout(
290             f"{timeout_exc} [cloud-run job={self._job_name} "
291             f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
"
292             "state in the GCP console]"
293         ) from timeout_exc
294
295     # --- storage-backed completion signal (bucket-poll) ---
296     def _read_marker(self, done_uri: str) -> Optional[dict]:
297         """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
298         ref = parse_uri(done_uri)
299         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
300             return None
301         return (
302             self._read_json(done_uri) or {}
303         ) # empty dict still terminates the poll (present == done)
304
305     def _read_json(self, uri: str) -> dict:
306         # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
307         # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
308         # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
309         try:
310             ref = parse_uri(uri)
311             with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
312                 dst = os.path.join(td, ref.name or "obj.json")
313                 local = fetch(uri, dst, config=self._storage_config)
314                 with open(local, "r", encoding="utf-8") as f:
315                     data: Any = json.load(f)
316                 return data if isinstance(data, dict) else {}
317         except Exception:
318             LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
319             return {}
320

```

10. assistant (2026-07-06T17:27:09.327Z)

Now let me read the `job_worker` consumer, focused on ``_resume_one_remote_job`` and ``_run_claimed_job``.

11. user (2026-07-06T17:27:10.258Z)

```

1     """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3     Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4     The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5     that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6     re-claimed when due ? with no central scheduler (the DB is the clock).
7
8     The monkeypatch contract these functions rely on (and that tests depend on):
9     every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10    `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11    `backend.main` module object at call-time (`import backend.main as _main`), never as a
12    bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13    ...)`` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
14    `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
15    inside the functions also avoids the circular import at module load (main.py imports
this
16    module; this module must not import main at import time).
17    """

```



```

18
19 import json
20 import logging
21 import os
22 import threading
23 import traceback
24 from concurrent.futures import ThreadPoolExecutor
25 from typing import Any, Dict, Optional
26
27 logger = logging.getLogger(__name__)
28
29 # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 by default: a single
30 # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
31 # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
32 # already parallelize their AI handoff internally via their own pools.
33 # EXCEPTION (#466): when the server CAN dispatch to Cloud Run (see
_cloud_dispatch_possible), a REMOTE
34 # job is only a bucket-poll (I/O-bound; GPU is on ephemeral L4s, WASB-only has no
35 # scales to deployment.max_concurrent_remote_jobs to overlap those. This does NOT
parallelize local
36 # work: CPU/GPU-heavy in-process detection + the compile stitch serialize via
37 # orchestration._LOCAL_COMPUTE_LANE regardless of the worker count, so the single-box
invariant holds
38 # and compile/local-override jobs stay serial. The count is fixed at first
_get_executor() creation.
39 #
40 # ? 02 RISK (CODE_CLEANUP_AUDIT §3c): a single worker means ONE stuck or hung job
41 # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
42 # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
43 # - ExecutionPolicy op_timeout_sec kills hung generate/processing calls
44 # - detector timeout_sec kills hung WSL/GPU calls
45 # - Cloud Run poll has a max_wait_sec bound
46 # - ffmpeg/ffprobe subprocesses use bounded media timeouts
47 # A future slice should add a per-job wall-clock timeout (the executor itself
48 # cannot enforce timeouts on the Thread).
49 _job_executor: Optional[ThreadPoolExecutor] = None
50 # Drain worker count, fixed at first _get_executor() creation. 1 (serial) by default;
configure_drain()
51 # raises it for cloud_run (#466). A module global (not a _get_executor arg) so the many
call sites +
52 # the tests that monkeypatch `_get_executor` as a no-arg lambda are unchanged.
53 _drain_max_workers: int = 1
54
55
56 def _cloud_dispatch_possible(config: Any = None) -> bool:
57     """True if this server CAN dispatch a job to Cloud Run ? either the profile sets
58     ``deployment.compute_target=cloud_run``, OR the per-request ``compute='cloud'``
override path is
59     wired (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT`` env + an output bucket). Mirrors
60     orchestration._cloud_available so drain concurrency tracks the ACTUAL remote-
dispatch capability,
61     not just the server default (so compute='cloud' overrides on a local-default server
parallelize too)."""
62     dep = getattr(config, "deployment", None)
63     if getattr(dep, "compute_target", "") == "cloud_run":
64         return True
65     if os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT"):
66         stor = getattr(config, "storage", None)
67         if (getattr(stor, "output_root", "") or "").strip():
68             return True
69     return False
70
71
72 def _resolve_drain_workers(config: Any = None) -> int:

```

```

73         """Drain concurrency (#466): a server that CAN dispatch to Cloud Run (see
       _cloud_dispatch_possible)
74         may run ``deployment.max_concurrent_remote_jobs`` drain workers so I/O-bound remote
dispatch/poll
75         OVERLAPS. This does NOT parallelize CPU/GPU-heavy LOCAL work ? in-process detection
and the compile
76         stitch serialize via orchestration._LOCAL_COMPUTE_LANE regardless of the worker
count ? so the
77         single-box GPU + rate-fragile Gemini invariant holds, and compile/local-override
jobs stay serial.
78         A server with no remote capability stays serial (1). Defaults to 1 when config is
absent
79         (tests / non-server) ? behaviour unchanged."""
80         if _cloud_dispatch_possible(config):
81             dep = getattr(config, "deployment", None)
82             return max(1, int(getattr(dep, "max_concurrent_remote_jobs", 1) or 1))
83         return 1
84
85
86     def configure_drain(config: Any = None) -> None:
87         """Set the drain worker count from ``config`` BEFORE the executor is created (#466).
Call once at
88         server startup. No-op effect once the executor already exists (the count is fixed at
creation)."""
89         global _drain_max_workers
90         _drain_max_workers = _resolve_drain_workers(config)
91
92
93     def _get_executor() -> ThreadPoolExecutor:
94         """Lazy module-global executor; tests monkeypatch this for inline execution. Worker
count is
95         ``_drain_max_workers`` (1 by default; raised for cloud_run via ``configure_drain``,
#466)."""
96         global _job_executor
97         if _job_executor is None:
98             _job_executor = ThreadPoolExecutor(
99                 max_workers=_drain_max_workers, thread_name_prefix="jobworker"
100             )
101         return _job_executor
102
103
104     class JobWaiting(Exception):
105         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
106         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`,
107         NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
108         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
109         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
110
111         The wiring is additive: the production segmenter path never raises this today (zero
behaviour change), so a job runs exactly as before. The hook is what a later slice
112         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
113         """
114
115
116         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
117             super().__init__(f"job parked for {delay_sec:.0f}s")
118             self.delay_sec = max(0.0, float(delay_sec))
119             self.progress = progress
120
121
122     def _job_to_request(job: Dict[str, Any]):
123         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
124         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to

```

```

125         the original submit. The row stores exactly the ProcessRequest fields."""
126         import backend.main as _main
127
128         raw_params = job.get("params")
129         params = (
130             raw_params if isinstance(raw_params, dict) else json.loads(raw_params or "{}")
131         )
132         return _main.ProcessRequest(
133             video_path=job["video_path"],
134             player_pool=job.get("player_pool"),
135             max_frames=job.get("max_frames"),
136             segmenter=job.get("segmenter"),
137             # v8 compute-picker: the worker runs _execute_processing ?
138             _maybe_dispatch_remote, which
139             # honours this, so it MUST survive submit?reconstruct (unlike locale, which is
140             # only read off
141             # the job row for analytics). NULL ? server default = pre-picker behaviour.
142             compute=job.get("compute"),
143             force=bool(params.get("force")),
144         )
145
146     def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
147         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
148         its
149         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
150
151         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
152         means
153         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
154         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
155         the
156         job self-scheduled and will be re-claimed when `next_poll_at` is due.
157         """
158         import backend.main as _main
159
160         job_id = job["id"]
161         # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
162         (NULL/default ? the
163         # original /api/process pipeline). Both record their terminal state the same way;
164         only process
165         # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
166         usage/segments).
167         is_compile = job.get("kind") == "compile"
168         try:
169             if is_compile:
170                 result = _main._execute_compile(
171                     config,
172                     db,
173                     job,
174                     progress=lambda stage: db.set_job_progress(job_id, stage),
175                 )
176             else:
177                 req = _main._job_to_request(job)
178                 result = _main._execute_processing(
179                     config,
180                     db,
181                     req,
182                     progress=lambda stage: db.set_job_progress(job_id, stage),
183                 )
184             if result.get("video_id"):
185                 db.set_job_video_id(job_id, result["video_id"])
186             db.mark_job_done(job_id, result)
187             if not is_compile:
188                 _maybe_record_job_cost(

```

```

182         job, result, config, db
183     ) # M8 cost ledger (default-OFF; process jobs only)
184     _maybe_run_flywheel(
185         job, result, config
186     ) # M11 data-flywheel (process jobs only)
187 except _main.JobWaiting as w:
188     # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
189     logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
190     db.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
191 except Exception as e:
192     logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
193     db.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
194
195
196 def _maybe_record_job_cost(
197     job: Dict[str, Any], result: Dict[str, Any], config: Any, db: Any
198 ) -> None:
199     """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
200     ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
201     (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
202     pricebook, and
203     persist it.
204     ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
205     so even with
206     ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
207     **0** until a
208     FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
209     ledger + pricebook
210     + the recording seam; the usage-emission step is separate (the metering hooks on the
211     real run).
212     With the placeholder pricebook the cost is 0 regardless.
213     Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
214     is the
215     claimed row (carries ``owner_id``); ``result`` is the pipeline output."""
216
217     billing_cfg = getattr(config, "billing", None)
218     if not billing_cfg or not getattr(billing_cfg, "enabled", False):
219         return
220     try:
221         usage = (result or {}).get("usage") or {}
222         db.record_job_cost(
223             job["id"],
224             usage=usage,
225             pricebook_version=billing_cfg.pricebook_version,
226             gpu_type=usage.get("gpu_type"),
227             compute_backend=(result or {}).get("compute_backend"),
228             owner_id=job.get("owner_id"),
229             margin=billing_cfg.margin,
230         )
231         logger.info(
232             "Recorded cost for job %s (pricebook=%s).",
233             job["id"],
234             billing_cfg.pricebook_version,
235         )
236     except Exception as e: # never wedge a completed job on bookkeeping
237         logger.warning(
238             "Cost recording for job %s failed (non-fatal): %s", job.get("id"), e
239         )
240
241 def _maybe_run_flywheel(
242     job: Dict[str, Any], result: Dict[str, Any], config: Any
243 ) -> None:

```

```

241         """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a
no-op unless
242         ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-
deny** today
243         (``flywheel.consent_attested`` returns False for every job until counsel + the
consent schema
244         land), so this captures **nothing** in production. When activated it captures the
consented job's
245         artifacts into the per-video workspace (? later labeling ? the golden training set).
246
247         Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is
the claimed
248         row (carries ``owner_id``); ``result`` is the pipeline output."""
249
250         flywheel_cfg = getattr(config, "flywheel", None)
251         if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
252             return
253         try:
254             from backend import flywheel
255
256             flywheel.run_for_job(job, result, config)
257         except Exception as e: # never wedge a completed job on the flywheel
258             logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)
259
260
261     def _drain_due_jobs(config: Any, db: Any) -> int:
262         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
263         ``next_poll_at`` is due), earliest-due first, until the table has nothing runnable.
264
265         Each parked job (``JobWaiting`` ? status 'waiting') drops out of this drain and is
picked
266         up by a FUTURE drain once due; we schedule that follow-up drain via
``_schedule_drain`` so
267         a single-worker box keeps making progress with no central timer (the DB is the
clock).
268
269         Returns the number of jobs that reached a terminal/parked state this tick.
270
271         """
272         import backend.main as _main
273
274         ran = 0
275         while True:
276             job = db.claim_due_job()
277             if job is None:
278                 break
279             ran += 1
280             _main._run_claimed_job(job, config, db)
281             # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
282             # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
283             # on the next submit/drain. Best-effort ? never raises into the worker.
284             _main._schedule_next_drain_if_waiting(config, db)
285             return ran
286
287     def _schedule_next_drain_if_waiting(config: Any, db: Any) -> None:
288         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
289         self-scheduled job resumes with no further client submit. The drain itself re-checks
due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
290         import backend.main as _main
291
292         try:

```

```

293         delay = db.seconds_until_next_due()
294     except Exception as e: # pragma: no cover - defensive; never wedge the worker
295         logger.error(f"Could not compute next-due delay: {e}")
296         return
297     if delay is None:
298         return # nothing waiting
299     # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
300     _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)), config, db)
301
302
303     def _arm_wake_timer(delay_sec: float, config: Any, db: Any) -> None:
304         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
305         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
306         can
307         monkeypatch it to a no-op and assert the *scheduling decision* without real
308         waiting."""
309         import backend.main as _main
310
311         timer = threading.Timer(
312             delay_sec,
313             lambda: _main._get_executor().submit(_main._drain_due_jobs, config, db),
314         )
315         timer.daemon = True
316         timer.start()
317
318     #: A parked job further out than this is re-checked by a capped wake rather than one
319     long
320     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
321     _MAX_DRAIN_WAKE_SEC = 60.0
322
323     #
324     -----
325     # Restart-safe remote jobs (CONTROL_PLANE_DURABLE_REBUILD P3, #471) ? the 2026-07-05
326     loss shape:
327     # a running cloud detection's done-marker poll lives only in this process's memory, so a
328     restart
329     # orphaned the job while the L4 worker kept computing; its 29 rallies landed in the
330     bucket with
331     # no listener. With the md5 persisted at submit (P4), outputs/<md5>/report.json IS a
332     durable
333     # done-signal the restarted control plane can re-arm on ? completing the job WITHOUT
334     ever
335     # re-dispatching (no double GPU spend).
336
337     #: Resume-poll cadence. Module-level so tests drive the loop with a zero-wait cadence.
338     _RESUME_POLL EVERY_SEC = 15.0
339
340
341     def resume_interrupted_remote_jobs(config: Any, db: Any) -> int:
342         """Re-arm the durable done-signal poll for md5-keyed RUNNING process jobs a restart
343         orphaned (the rows ``recover_stale_jobs`` deliberately left alone). Call once at
344         startup,
345         after ``recover_stale_jobs`` and before any drain. Each job resumes on its own
346         daemon
347         thread (bounded by the handful of rows a single-instance CP can have in flight);
348         rows are
349         NEVER re-dispatched from here. Returns the number of resumes started.
350
351         On a box that cannot dispatch to Cloud Run at all (the appliance), no remote worker
352         can
353         exist ? the rows get today's honest terminal failure instead."""
354         rows = db.interrupted_remote_process_jobs()

```

```

344     if not rows:
345         return 0
346     if not _cloud_dispatch_possible(config):
347         for job in rows:
348             db.mark_job_failed(str(job["id"]), "interrupted by server restart")
349             logger.warning(
350                 "[JOBS] %d interrupted md5-keyed process job(s) failed: this server cannot "
351                 "dispatch to Cloud Run, so no remote worker can be in flight.",
352                 len(rows),
353             )
354         return 0
355     # Prove each row was actually CLOUD-dispatched (#494 review round 2): an explicit
356     # compute='cloud' always was; NULL means "server default", which is remote ONLY when
the
357     # default target itself is cloud_run (on a default-local server with cloud-override
358     # capability, a NULL row ran in-process ? no remote worker exists to resume).
359     default_is_cloud = (
360         getattr(getattr(config, "deployment", None), "compute_target", "") ==
"cloud_run"
361     )
362     started = 0
363     for job in rows:
364         # Same normalization the execution path applies (_maybe_dispatch_remote
lowercases
365         # req.compute) ? "LOCAL"/" cloud " must mean at recovery exactly what they meant
at run.
366         requested = (str(job.get("compute") or "")).strip().lower()
367         if requested != "cloud" and not default_is_cloud:
368             db.mark_job_failed(str(job["id"]), "interrupted by server restart")
369             continue
370         t = threading.Thread(
371             target=_resume_one_remote_job,
372             args=(config, db, dict(job)),
373             daemon=True,
374             name=f"resume-{str(job['id'][:8])}",
375         )
376         t.start()
377         started += 1
378     if started:
379         logger.warning(
380             "[JOBS] re-armed the durable done-signal poll for %d interrupted remote
job(s).",
381             started,
382         )
383     return started
384
385
386 def _resume_one_remote_job(config: Any, db: Any, job: Dict[str, Any]) -> None:
387     """One orphaned job's resume: poll ``outputs/<md5>/report.json`` (bounded by the
same
388     ``deployment.remote_poll_max_wait_sec`` budget a live dispatch gets), then complete
or
389     fail the job HONESTLY from what the bucket says. Never dispatches."""
390     from backend import orchestration, storage
391     from backend.infrastructure.execution_policy import (
392         ExecutionPolicy,
393         PolicyTimeout,
394         poll_until,
395     )
396
397     job_id = str(job["id"])
398     video_id = str(job["video_id"])
399     try:
400         raw_params = job.get("params")
401         params = (

```

```

402         raw_params
403         if isinstance(raw_params, dict)
404         else json.loads(raw_params or "{}")
405     )
406     if params.get("force"):
407         # Defense-in-depth (#494 review): the SQL carve-out already excludes forced
rows,
408         # but a forced job must NEVER complete from the bucket cache ? force
deliberately
409         # bypasses it, and nothing proves the report belongs to the forced run
rather
410         # than an older completed one.
411         db.mark_job_failed(
412             job_id,
413             "control plane restarted during a forced reprocess; the cached result "
414             "cannot be trusted for force=true ? re-submit with force",
415         )
416         return
417     if (str(job.get("compute") or "").strip().lower() == "local":
418         # Defense-in-depth (#494 review round 2): an explicit local run has no
remote
419         # worker; a same-md5 bucket report would be an OLDER run's output, not this
job's.
420         db.mark_job_failed(job_id, "interrupted by server restart")
421         return
422     sc = orchestration._storage_settings(config)
423     output_root = ((sc or {}).get("output_root") or "").rstrip("/")
424     if not sc or not output_root:
425         db.mark_job_failed(
426             job_id,
427             "interrupted by server restart (no output bucket configured to recover
from)",
428         )
429         return
430     report_uri = f"{output_root}/outputs/{video_id}/report.json"
431     max_wait = float(
432         getattr(getattr(config, "deployment", None), "remote_poll_max_wait_sec",
3900.0)
433         or 3900.0
434     )
435     policy = ExecutionPolicy(
436         poll_every_n_sec=_RESUME_POLL_EVERY_SEC,
437         op_timeout_sec=120.0,
438         max_wait_sec=max_wait,
439     )
440     db.set_job_progress(job_id, "recovering: waiting for the in-flight cloud run")
441     try:
442         poll_until(
443             lambda: True if storage.exists(report_uri, config=sc) else None,
444             policy,
445             label=f"resume-{job_id[:8]}",
446             logger=logger,
447             # The #493 live heartbeat: job.progress moves once per still-waiting
tick.
448             on_tick=lambda elapsed: db.set_job_progress(
449                 job_id,
450                 "recovering: waiting for the in-flight cloud run ? "
451                 f"{int(elapsed) // 60} min",
452             ),
453         )
454     except PolicyTimeout:
455         db.mark_job_failed(
456             job_id,
457             "control plane restarted mid-run and the cloud worker's result never "
458             "appeared within the poll budget; re-submit the video (a finished result

```



```

"
459         "re-hydrates instantly)",
460     )
461     return
462     payload = orchestration.cached_process_result(
463         db, config, video_id, str(job.get("video_path") or "")
464     )
465     if payload is not None:
466         db.mark_job_done(job_id, payload)
467         logger.warning(
468             "[JOBS] resumed job %s: ingested the in-flight worker's result for %s.",
469             job_id,
470             video_id,
471         )
472     return
473     # The report exists but did not realize into a usable result ? surface its
474     verdict.
475     status = "unreadable"
476     try:
477         report = orchestration._read_bucket_json(report_uri, sc) or {}
478         status = str(report.get("status") or "unreadable")
479     except Exception: # noqa: BLE001 ? the failure message is best-effort
480         pass
481     db.mark_job_failed(
482         job_id,
483         f"cloud worker finished without a usable result after a control-plane
484         restart "
485         f"(report status: {status}); re-submit the video",
486     )
487     except Exception as e: # noqa: BLE001 ? a resume crash must land as a job failure,
488     not vanish
489     logger.error("[JOBS] resume of job %s crashed: %s", job_id, e, exc_info=True)
490     try:
491         db.mark_job_failed(job_id, f"restart recovery failed: {e}")
492     except Exception: # noqa: BLE001
493         pass

```

12. assistant (2026-07-06T17:27:17.213Z)

Now I have a strong picture. Let me look at a critical interaction: ``probe_process_cache`` and ``cached_process_result`` gate on ``status == "success"``, but the fix now writes a ``status="failed"`` report.json at ``outputs/<md5>/report.json`` for the SAME md5. Let me check how a re-submit of the same md5 interacts, and examine the compute path in ``main.py`/orchestration` for how a cloud dispatch result is consumed. Let me look at how the dispatch result feeds back (main.py isinstance Failure branch and ``_ingest_remote_segments``, and the compute result consumer).

13. user (2026-07-06T17:27:19.840Z)

```

241:     _maybe_dispatch_remote,
268:     # default (deployment.compute_target). Only honoured for these two values; see
    _maybe_dispatch_remote.

```

14. user (2026-07-06T17:27:21.207Z)

```

101:     if report.get("output_dir"):
103:         report["output_dir"] = os.path.join(output_base(config), "reports")
217:         "status": "success",
255: def probe_process_cache(db, cfg, video_id: str) -> bool:
280:     return bool(report and report.get("status") == "success")
288: def cached_process_result(
292:     the DB first, then the bucket (`outputs/<md5>/report.json` with `status=success`
326:     if not report or report.get("status") != "success":

```

```

334:         video_path = str(report.get("video_uri") or "") or _first_object_under(
386:     if not probe_process_cache(db, cfg, vid):
388:     return cached_process_result(db, cfg, vid, "") is not None
424:         "status": "failed",
571:     if result.returncode != 0:
572:         return _failed(f"cloud job failed (returncode {result.returncode}).", True)
624:         "status": "success",
743:         "status": "failed",
762:         "status": "failed",
811:         "status": "failed",
824:         "status": "success",
994:     ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
1010:         return {"status": "failed", "message": e.detail, "video_id": video_id}
1045:         "status": "failed",
1074:         "status": "success",

```

15. assistant (2026-07-06T17:27:28.098Z)

Let me look at the `\_maybe\_dispatch\_remote` flow in orchestration around line 560-640, where the ComputeResult is consumed and the returncode gate lives.

16. user (2026-07-06T17:27:29.030Z)

```

520             f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
521             False,
522         )
523         # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
524         # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
525         # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
526         req.video_path = staged_uri
527         input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
528         db.add_video(
529             video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
530         )
531         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
532         if not out_root:
533             return _failed(
534                 "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
535                 "? that's where the Cloud Run job writes the result.",
536                 False,
537             )
538
539         notify("dispatching (cloud_run)")
540         logger.info(
541             "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
542             req.video_path,
543             out_root,
544         )
545         # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
546         # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
547         # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
548         # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
549         overrides = []
550         sk = getattr(getattr(cfg, "indexing", None), "skip_ai_handoff", None)
551         if sk is not None:

```

```

552         overrides.append(f"indexing.skip_ai_handoff={ 'true' if sk else 'false' }")
553     spec = ComputeJobSpec(
554         video_uri=req.video_path,
555         out_uri=out_root,
556         segmenter=seg_name,
557         config_overrides=tuple(overrides),
558     )
559     try:
560         # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
561         # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
562         # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
563         # also gives the SPA a stall signal to time out on, instead of a blind wall
564         result = compute.run_infer(
565             spec,
566             on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
567         )
568     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
569         logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
570         return _failed(f"cloud dispatch error: {e}", True)
571     if result.returncode != 0:
572         return _failed(f"cloud job failed (returncode {result.returncode}).", True)
573     if result.video_id and result.video_id != video_id:
574         # The worker re-hashes the bucket bytes; a divergence means the input changed
575         # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
576         # diagnosable.
577         logger.warning(
578             "[API] cloud video_id %s != local %s ? input bytes differ between the API
579             hash "
580             "and the worker hash; ingesting under the local id.",
581             result.video_id,
582             video_id,
583         )
584     notify("ingesting (cloud_run)")
585     storage_dict = (
586         storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
587         if hasattr(storage_cfg, "model_dump")
588         else {}
589     )
590     try:
591         _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
592     except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
593         logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
594         return _failed(
595             f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
596             True,
597         )
598     # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
599     # drops the old ones.
600     segments = [
601         s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
602     ]
603     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
604     # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
605     # is the real
606     # Cloud Run execution name (cross-check: gcloud run jobs executions describe
607     # <execution>).
608     provenance = {

```

```

606         "path": "cloud_run",
607         "requested": req.compute,
608         "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
609         "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
610         "execution": result.execution,
611         "outputs_uri": result.outputs_uri,
612     }
613     logger.info(
614         "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
615         result.execution,
616         provenance["job"],
617         provenance["region"],
618         result.outputs_uri,
619         video_id,
620     )
621     return (
622         {
623             "video_id": video_id,
624             "status": "success",
625             "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
626             "results": [r.model_dump() for r in val_results],
627             "play_segments": segments,
628             "compute": provenance,
629         },
630         True,
631     )
632
633
634     def _execute_processing(
635         config=None, db=None, req=None, progress=None
636     ) -> Dict[str, Any]:
637         """The full hash ? validate ? segment ? report pipeline for one video.
638
639         Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
640         module-level ``global_config``/``db_instance`` (backward-compatible with existing
641         tests that pass only ``req``). Route handlers always pass explicit params; older
642         test code that calls ``_execute_processing(req)`` still works.
643
644         ``progress`` is an optional callable(stage: str) used to surface coarse job
645         progress.
646         Raises on unexpected internal errors ? the caller records those as a job failure
647         """
648         import backend.main as _main # call-time seam: main's globals/model stay patchable
649         # Backward-compatible shim: if called as _execute_processing(req) (old pattern),

```

17. assistant (2026-07-06T17:27:38.669Z)

The cloud path returncode gate (`result.returncode != 0 ? _failed`) is correct: a failed report + rc=3 marker ? `_failed`. Now the key concern: does writing a `status="failed"` report.json at `outputs/<md5>/report.json` block or corrupt a later legitimate re-submit of the same md5?

Both `probe_process_cache` (line 280) and `cached_process_result` (line 326) gate on `status == "success"` ? a "failed" report returns `None`/`False`, correctly a cache miss ? normal dispatch. That's fine. But there's a real concern for the cloud dispatch path on re-submit. When a genuine 0-rally video is re-submitted after a prior detector timeout wrote a `status="failed"` report, the new run will overwrite it. Let me verify there's no ordering issue where the OLD failed report is read.

Let me check `_maybe_dispatch_remote` entry point and how `cached_process_result`/probe are called before dispatch, and whether the cloud dispatch consumes the report status (vs just returncode). Let me look at the top of `_maybe_dispatch_remote`.

18. user (2026-07-06T17:27:40.285Z)

```
470         except ValueError as e:
471             return _failed(
472                 f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
473             )
474         if input_ref.backend == "local":
475             # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
476             # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
477             # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
478             # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
479             # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
480             # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
481             # default, while a bare/local-looking root can't stage no matter what
input_backend says
482             # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
483             # gives us nothing to build the staged URI under).
484             input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
485             try:
486                 root_backend = (
487                     storage.parse_uri(input_root).backend if input_root else "local"
488                 )
489             except ValueError: # unsupported scheme in input_root ? nowhere real to stage
to
490                 root_backend = "local"
491             if root_backend == "local":
492                 return _failed(
493                     "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
494                     "stage local uploads into; a path local to this machine can't be read by
the Cloud "
495                     "Run job.",
496                     False,
497                 )
498             local_src = input_ref.key # resolved local path of the upload
499             staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
500             try:
501                 notify("staging upload to bucket")
502                 # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
503                 # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
default client.
504                 storage.put(
505                     local_src,
506                     staged_uri,
507                     config=(
508                         storage_cfg.model_dump() if storage_cfg is not None else None
509                     ),
510                 )
511             except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
512                 logger.error(
513                     "[API] cloud-serving: failed to stage %s -> %s: %s",
514                     local_src,
515                     staged_uri,
516                     e,
517                     exc_info=True,
```

```

518         )
519         return _failed(
520             f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
521             False,
522         )
523         # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
524         # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is

```

19. assistant (2026-07-06T17:27:51.768Z)

Now let me check the critical interaction between the cloud dispatch and `\_ingest\_remote\_segments`. On the cloud path, a returncode != 0 correctly triggers `\_failed` (line 571-572). But there is a subtle interaction worth verifying: the done-marker in `\_read\_marker` returns an empty dict if the marker exists but is empty. Let me now focus on the most important cross-check the task flags: the `\_resume\_one\_remote\_job` path and how it interacts with the new failed report.

Let me trace the `\_resume\_one\_remote\_job` flow carefully against the new behavior. Previously, a detector-timeout wrote NO report.json (the abort returned bare `[]`, but wait ? previously it wrote a false SUCCESS report). Let me re-check: the diff says previously the detector abort returned bare `[]` which led to a `status="success"` report with 0 segments. Now it returns a `Failure` ? `\_fail(3)` ? `status="failed"` report.

For `\_resume\_one\_remote\_job`: it polls for `report.json` to EXIST (line 442-443), then calls `cached\_process\_result` (line 462). With a failed report, `cached\_process\_result` returns `None` (status != success), so it falls to line 473-484: reads the report status ("failed") and calls `mark\_job\_failed`. This is strictly better than before. Good.

Now let me examine a genuine concurrency/idempotency concern: the ordering of pushes in `\_serialize\_and\_push\_outputs` vs the done-marker, and the resume poll's race. Let me check the parallel `yolo\_hybrid.py` segmenter and confirm whether it too can time out but still returns bare `[]` (a remaining silent-failure leak, but that's out of scope unless it's a regression). Let me verify the exact push ordering concern with `\_resume\_one\_remote\_job`.

20. user (2026-07-06T17:27:53.827Z)

```

255         inactivity_min_density = idx_cfg.inactivity_min_density
256         inactivity_temporal_density = idx_cfg.inactivity_temporal_density
257         window_blob_fallback = idx_cfg.window_blob_fallback
258         window_blob_factor = idx_cfg.window_blob_factor
259         handoff_padding = idx_cfg.ai_handoff_padding
260         ai_max_workers = idx_cfg.ai_max_workers
261         log_candidates = idx_cfg.log_yolo_candidates
262         store_candidates = idx_cfg.store_yolo_candidates
263
264         detection_params = {
265             "detector": self.name,
266             "velocity_thresh": velocity_thresh,
267             "merge_gap": merge_gap,
268             "min_action_window_duration": min_window_duration,
269             **runner_params,
270         }
271
272         # --- Phase 1: env healthcheck (fail fast with a clear message) ---
273         ok, msg = runner.healthcheck()
274         if not ok:
275             logger.error("%s detector environment not ready: %s", label, msg)
276             # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
277             # failed/retryable instead of writing an empty-but-"successful" result.
278             return Failure(
279                 message=f"{label} detector environment not ready: {msg}",

```

```

280         is_recoverable=True,
281     )
282     logger.info("%s detector env OK: %s", label, msg)
283
284     # --- Phase 2: trajectory -> candidate rally windows ---
285     # Trajectory detectors process the whole video in one pass (they carry
286     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
287     t_start = time.time()
288     out_dir = os.path.join(output_base(self.config), self.family)
289     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
290     if not csv_path:
291         logger.error("%s produced no trajectory; aborting.", label)
292         # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
293         # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
294         # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
295         # rather than a silent 0-segment "success" (the wf294 incident).
296         return Failure(
297             message=(
298                 f"{label} detector produced no trajectory (timed out or aborted) ? "
299                 "detector failure, not a 0-rally video."
300             ),
301             is_recoverable=True,
302         )
303
304     points = runner.parse_trajectory_csv(csv_path)
305     logger.info(
306         "Parsed %d trajectory points (%d visible).",
307         len(points),
308         sum(1 for p in points if p.visible),
309     )
310
311     all_candidate_windows = runner.trajectory_to_action_windows(
312         points,
313         fps=fps,
314         frame_width=frame_width,
315         chunk_start=0.0,
316         velocity_thresh=velocity_thresh,
317         merge_gap=merge_gap,
318         min_window_duration=min_window_duration,
319         chunk_index=0,
320         max_window_duration=max_window_duration,
321         min_split_gap=window_min_split_gap,
322         window_hard_cap=window_hard_cap,
323         window_inactivity_end=window_inactivity_end,
324         inactivity_rally_thresh=inactivity_rally_thresh,

```

21. assistant (2026-07-06T17:27:58.409Z)

Now let me check the `main.py` inline (non-cloud) path ? how it handles a `Failure` from the segmenter, to confirm the fix is consistent between the worker path and the direct main.py path. The task mentioned `backend/main.py` isinstance Failure branch ~1557.

22. user (2026-07-06T17:27:59.716Z)

```

1530         print("\n" + "=" * 40)
1531         print("VALIDATION SUMMARY")
1532         print("=" * 40)
1533         for r in val_results:
1534             status_symbol = "[PASS]" if r.passed else "[FAIL]"
1535             print(f"{status_symbol} {r.validator_name}: {r.message}")
1536         print("=" * 40 + "\n")
1537

```

```

1538         if failures:
1539             print("[FAIL] Video failed checks. Indexing halted.")
1540             return
1541
1542         # Index
1543         segmenter_cls = segmenter_registry.get(seg_name)
1544         if not segmenter_cls:
1545             print(f"[FAIL] Segmenter '{seg_name}' not found.")
1546             return
1547
1548         print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
1549         segmenter_actual = seg_name
1550         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
1551         segmenter = segmenter_cls(db_instance, global_config)
1552         result = segmenter.process_video(
1553             video_id, local_video, max_frames=args.max_frames
1554         )
1555         segmentation_ran = True
1556
1557         if isinstance(result, Failure):
1558             print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
1559             print("[FAIL] Indexing halted for video.")
1560             segments = []
1561         else:
1562             segments = result.value if hasattr(result, "value") else result
1563             print(
1564                 f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}"
1565             )
1566             # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
1567             _finalize_reingest(db_instance, video_id, segments, play_wm, cand_wm)
1568         finally:
1569             # Always emit the per-video observability report. Never raises.
1570             paths = emit_indexer_report(
1571                 db=db_instance,
1572                 video_id=video_id,
1573                 # Resolved LOCAL path, not the raw URI (see #397) ? a remote URI
would write a
1574                 # literal scheme dir (gcs:/, gs:/, local:/) at the repo root. Source
URI is kept
1575                 # in DB provenance via db.add_video.
1576                 video_path=local_video,
1577                 config=_report_config_for_input(global_config, args.video_path),
1578                 val_results=val_results,
1579                 val_context=val_context,
1580                 segmenter_requested=args.segmenter,
1581                 segmenter_actual=segmenter_actual,
1582                 was_reingest=was_reingest,
1583                 segmentation_ran=segmentation_ran,
1584             )
1585             if paths:
1586                 print(f"[REPORT] Technical report : {paths.get('technical_md')}")
1587                 print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
1588                 print(f"[REPORT] Machine JSON      : {paths.get('json')}")
1589         finally:
1590             storage.cleanup_acquired_local_input(
1591                 args.video_path, local_video, storage_cfg
1592             )
1593         return
1594
1595     # Start Local Server Mode
1596     if args.app or args.server or not args.video_path:
1597         if args.app:
1598             # P2: the friendly launcher ? always localhost-only (use --server for

```



```
network exposure),
1599         # auto-pick a free port if --port is busy, and open the bundled UI once the
server is up.
```

23. assistant (2026-07-06T17:28:06.615Z)

Now let me check the `\_execute\_processing` inline path (the API/job path, not the CLI) ? how the segmenter Failure is handled there, since that's what the job_worker calls. This is where a Failure needs to route to `status="failed"`.

24. user (2026-07-06T17:28:07.972Z)

```
700         with open(path, "w", encoding="utf-8", newline="") as f:
701             f.write(req.csv or "")
702         # Count CSV data rows (a line starting with an integer rally_number) for an honest
ack.
703         num_rows = sum(
704             1 for ln in (req.csv or "").splitlines() if ln[:1].isdigit() and "," in ln
705         )
706         logger.info(
707             "annotations saved: video_id=%s rows=%d -> %s", req.video_id, num_rows, path
708         )
709         return {"status": "ok", "num_rows": num_rows, "csv_uri": f"labels/{name}"}
710
711
712     def _storage_settings(cfg) -> Dict[str, Any]:
713         """The storage config as a plain settings dict (NO secrets ? see StorageConfig)."""
714         sc = getattr(cfg, "storage", None)
715         if sc is None:
716             return {}
717         return sc.model_dump() if hasattr(sc, "model_dump") else dict(sc)
718
719
720     def _asset_dest_uri(medium_uri: str, video_id: str, ext: str) -> str:
721         """The durable, MD5-keyed destination inside the chosen medium:
`<medium>/<video_id><ext>`. """
722         return f"{medium_uri.rstrip('/')}/{video_id}{ext}"
723
724
725     def _medium_local_dir(medium_ref, storage_settings: Dict[str, Any]) -> Optional[str]:
726         """The LOCAL directory a ``put`` into this medium would write into (so a free-space
BLOCK can be
727         enforced, D11), or ``None`` for a cloud bucket (which is cost/quota-warned, never
blocked).
728         ``local`` ? the folder; ``gdrive`` ? the resolved path under the Drive mount."""
729         if medium_ref.backend == "local":
730             return medium_ref.key or "."
731         if medium_ref.backend == "gdrive":
732             from backend.storage.capabilities import _gdrive_mount_root
733
734             root = _gdrive_mount_root(storage_settings)
735             if not root:
736                 return None
737             rel = medium_ref.key.lstrip("/")
738             return os.path.join(root, *rel.split("/")) if rel else root
739         return None # s3 / gcs ? a bucket has no readable local capacity
740
741
742     @app.post("/api/assets", status_code=201)
743     def create_asset(req: AssetRequest, request: Request):
744         """Store a video into the user's chosen medium and register it as a library asset
(P4, D14).
745
746         SELF-HOST posture only (D7): refused with 403 when ``security.allowed_video_root``
is set ? the
```

```

747         hosted alpha jails inputs and has no per-user medium credentials, so it stays
upload-to-local.
748         Edge-auth mirrors ``/api/process``. Exactly one source: an ``uploaded_path`` (a
local file from
749         the chunked upload) OR a ``source_uri`` (a video already at rest in a medium). An
uploaded file +
750         a REMOTE medium ? a durable, MD5-keyed copy is ``put`` into the medium while the
local upload
751         stays as the working copy; a ``local`` medium keeps the upload in place (it's
already on this box);
752         a pasted at-rest URI is registered as-is (no copy, D14b). The asset is keyed by the
whole-file
753         MD5 (``compute_video_id``, D3) so it joins the pipeline by content hash, and appears
in
754         ``GET /api/videos``.
755     try:
756         owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
757     except edge_auth.EdgeAuthError as e:
758         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
759
760     cfg = request.app.state.config
761     sec = getattr(cfg, "security", None)
762     if getattr(sec, "allowed_video_root", None):
763         raise HTTPException(
764             status_code=403,
765             detail=(
766                 "storing into a custom medium is disabled on this hosted deployment "
767                 "(security.allowed_video_root is set); upload to local instead."
768             ),
769         )
770     if bool(req.uploaded_path) == bool(req.source_uri):
771         raise HTTPException(
772             status_code=400,
773             detail="provide exactly one of 'uploaded_path' or 'source_uri'.",
774         )
775     if req.uploaded_path and not req.medium_uri:
776         raise HTTPException(
777             status_code=400, detail="'medium_uri' is required when uploading a file."
778         )
779
780     storage_settings = _storage_settings(cfg)
781
782     # 1) Resolve a LOCAL working copy of the source + its whole-file MD5 (the join key,
D3).
783     if req.uploaded_path:
784         try:
785             validate_input_path(req.uploaded_path, allowed_root=None)
786             src_ref = storage.resolve_input_ref(req.uploaded_path, storage_settings)
787         except ValueError as e:
788             raise HTTPException(status_code=400, detail=str(e)) from e
789         if src_ref.backend != "local" or not os.path.isfile(src_ref.key):
790             raise HTTPException(
791                 status_code=404,
792                 detail=f"uploaded file not found at '{req.uploaded_path}'",
793             )
794         local_working = src_ref.key
795     else:
796         try:
797             validate_input_path(req.source_uri, allowed_root=None)
798             src_ref = storage.resolve_input_ref(req.source_uri, storage_settings)
799         except ValueError as e:
800             raise HTTPException(status_code=400, detail=str(e)) from e
801         # A remote at-rest source is fetched to scratch to hash it (D3) ? reserve that
scratch (P2).
802         if src_ref.backend != "local":

```

```

803         need = _remote_input_size_bytes(src_ref, getattr(cfg, "storage", None))
804         require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
805     try:
806         local_working = storage.acquire_local_input(
807             req.source_uri, getattr(cfg, "storage", None)
808         )
809     except ValueError as e:
810         raise HTTPException(status_code=400, detail=str(e)) from e
811     except Exception as e: # noqa: BLE001 ? a fetch miss is a 502, surfaced not
swallowed
812         raise HTTPException(
813             status_code=502, detail=f"could not resolve source_uri: {e}"
814         ) from e
815     if not os.path.isfile(local_working):
816         raise HTTPException(
817             status_code=404, detail="source_uri did not resolve to a readable file."
818         )
819
820     cleanup_raw = req.source_uri or ""
821     try:
822         video_id = compute_video_id(local_working)
823         ext = os.path.splitext(local_working)[1] or ".mp4"
824
825         # 2) Decide the durable location + copy only when needed.
826         copied = False
827         if req.source_uri:
828             stored_uri = (
829                 req.source_uri

```

25. assistant (2026-07-06T17:28:15.320Z)

Let me find the local (non-cloud) inline segmenter execution in `\_execute\_processing` where the Failure is handled ? around lines 990-1075 based on the earlier grep.

26. user (2026-07-06T17:28:16.871Z)

```

985     fx = request.app.state.config.billing.fx_inr_per_usd
986     usd = cost.get("cost_usd")
987     cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
988     cost["fx_inr_per_usd"] = fx
989     return cost
990
991
992     @app.get("/api/videos/{video_id}/cost")
993     def get_video_cost(video_id: str, request: Request):
994         """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
995         ``total_cost_inr`` + the
several."""
996         agg = request.app.state.db.get_video_cost(video_id)
997         fx = request.app.state.config.billing.fx_inr_per_usd
998         agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
999         agg["fx_inr_per_usd"] = fx
1000         return agg
1001
1002
1003     @app.get("/api/jobs/{job_id}")
1004     def get_job(job_id: str, request: Request):
1005         """Job state: {status, progress, result, reports}. `status` = execution state
1006         (queued|running|done|failed); the pipeline verdict is result["status"]."""
1007         job = request.app.state.db.get_job(job_id)
1008         if not job:
1009             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
1010         result = job.get("result")
1011         return {

```

```

1012         "job_id": job["id"],
1013         "status": job["status"],
1014         "progress": job.get("progress"),
1015         "video_id": job.get("video_id"),
1016         "error": job.get("error"),
1017         "result": result,
1018         "reports": (result or {}).get("reports"),
1019         "created_at": job.get("created_at"),
1020         "started_at": job.get("started_at"),
1021         "finished_at": job.get("finished_at"),
1022     }
1023
1024
1025     app.include_router(jobs_api.router) # GET /api/jobs ? the processing-queue view (#448)
1026
1027
1028     # --- Chunked upload (B2, PR-2) -----
1029     # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
1030     # in
1031     # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
1032     # via
1033     # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
1034     # sequential-part logic, and abort cleanup ? is unchanged.
1035     from backend.api import uploads as uploads_api
1036
1037     app.include_router(uploads_api.router)
1038
1039
1040     # --- Highlight download (B3, PR-2) -----
1041
1042     @app.get("/api/highlights/{video_id}/{filename}")
1043     def download_highlight(video_id: str, filename: str, request: Request):
1044         """Stream a compiled highlight reel ? `filename` is the bare name given to
1045         /api/compile
1046         (which only writes into output_dir; the browser previously got back an unreachable
1047         server-local path)."""
1048         try:
1049             name = safe_output_filename(filename)
1050             except ValueError as e:
1051                 raise HTTPException(status_code=400, detail=str(e)) from e
1052             if not request.app.state.db.get_video(video_id):
1053                 raise HTTPException(status_code=404, detail="Indexed video record not found.")
1054             # #463: when the reel was mirrored to the output bucket, 307 to a short-lived signed
1055             URL (bytes flow
1056             # client<->store, bypassing Cloud Run's ~32 MiB response cap + the ephemeral local
1057             /tmp). The
1058             # exists/sign logic (with the active storage config) lives in
1059             orchestration.signed_reel_url to keep
1060             # this module lean; best-effort ? any error falls back to the local file below.
1061             try:
1062                 signed = signed_reel_url(request.app.state.config, video_id, name)
1063                 if signed:
1064                     return RedirectResponse(signed, status_code=307)
1065             except Exception as e: # noqa: BLE001 ? signing/exists hiccup ? serve the local
1066             file instead
1067                 logger.warning("[highlights] signed-url path failed for %s; serving local: %s",
1068                 video_id, e)
1069             output_dir = (
1070                 getattr(
1071                     getattr(request.app.state.config, "output", None), "output_dir", "output"
1072                 )
1073                 or "output"
1074             )
1075             path = os.path.join(output_dir, name)

```

```

1069         try:
1070             path = resolve_under_root(path, output_dir)
1071         except ValueError as e:
1072             raise HTTPException(status_code=400, detail=str(e)) from e
1073     if not os.path.isfile(path):
1074         raise HTTPException(
1075             status_code=404,
1076             detail=f"No compiled file named '{name}'. Compile first via /api/compile.",
1077         )
1078     media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
1079     return FileResponse(path, media_type=media, filename=name)

```

27. assistant (2026-07-06T17:28:19.784Z)

That's not the segmenter execution. Let me find the actual local segmenter execution inside `\_execute\_processing`/`\_run\_local\_processing` where isinstance Failure is checked in the API path.

28. user (2026-07-06T17:28:22.090Z)

```

905: def process_video(req: ProcessRequest, request: Request):
1539:     print("[FAIL] Video failed checks. Indexing halted.")
1552:     result = segmenter.process_video(
1557:         if isinstance(result, Failure):
1559:             print("[FAIL] Indexing halted for video.")

```

29. assistant (2026-07-06T17:28:28.813Z)

The API/job local path doesn't call `process\_video` directly in main.py ? it must be in orchestration `\_execute\_processing`. Let me search there.

30. user (2026-07-06T17:28:30.558Z)

```

280:         return bool(report and report.get("status") == "success")
292:     the DB first, then the bucket (``outputs/<md5>/report.json`` with ``status=success``
326:         if not report or report.get("status") != "success":
795:             result = segmenter.process_video(
806:             if isinstance(result, Failure):
994:         ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)

```

31. user (2026-07-06T17:28:33.302Z)

```

730         failures = [r for r in val_results if not r.passed]
731
732         if failures:
733             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
734             # status; it no longer destroys the video's prior good segments.
735             db.add_video(
736                 video_id,
737                 req.video_path,
738                 "failed",
739                 [r.model_dump() for r in val_results],
740             )
741             response = {
742                 "video_id": video_id,
743                 "status": "failed",
744                 "message": _validation_failure_message(failures),
745                 "results": [r.model_dump() for r in val_results],
746             }
747             return response
748
749         # 2. Run segmenter & indexer
750         logger.info("[API] Video passed checks. Segmenting and indexing...")

```

```

751         db.add_video(
752             video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
753         )
754
755     segmenter_cls = segmenter_registry.get(seg_name)
756     if not segmenter_cls:
757         # Submit-time validation already 400s unknown names; this is the defensive
758         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
759         available = list(segmenter_registry.list_available().keys())
760         response = {
761             "video_id": video_id,
762             "status": "failed",
763             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
764             "results": [r.model_dump() for r in val_results],
765             "play_segments": [],
766         }
767         return response
768
769     logger.info(f"[API] Using segmenter: {seg_name}")
770     notify(f"segmenting ({seg_name})")
771     segmenter_actual = seg_name
772     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
773     # if the run yields segments; otherwise drop the new partial rows and keep the
old.
774     play_wm, cand_wm = db.segment_watermarks(video_id)
775     # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
776     # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
777     # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
778     # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
779     # they're dropped on the cloud path in this slice ? a documented follow-up.)
780     remote = _maybe_dispatch_remote(
781         config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
782     )
783     if remote is not None:
784         response, segmentation_ran = (
785             remote # `ran` = did the cloud job actually execute (report)
786         )
787         return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
788     # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
789     compute_actual = "in_process"
790     segmenter = segmenter_cls(db, config)
791     try:
792         # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
793         # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
794         with _LOCAL_COMPUTE_LANE:
795             result = segmenter.process_video(
796                 video_id, local_video, req.player_pool, req.max_frames
797             )
798     except Exception:
799         # A raising segmenter must not leave half-written rows: restore the pre-run
800         # state (same destroy-after-confirm contract as the Failure branch), then
let
801         # the job worker record the failure.
802         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)

```

```

803         raise
804     segmentation_ran = True
805
806     if isinstance(result, Failure):
807         logger.error(f"[API] Segmenter failed: {result.message}")
808         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
809         response = {
810             "video_id": video_id,
811             "status": "failed",
812             "message": f"Segmenter '{seg_name}' failed: {result.message}",
813             "results": [r.model_dump() for r in val_results],
814             "play_segments": [],
815         }
816         return response
817
818     segments = result.value if hasattr(result, "value") else result
819     notify("finalizing")
820     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
821
822     response = {
823         "video_id": video_id,
824         "status": "success",
825         "message": f"Successfully indexed {len(segments)} play segments using
'[{seg_name}]' segmenter.", # type: ignore[arg-type] # hasattr-unwrap above; Success arm has
.value
826         "results": [r.model_dump() for r in val_results],
827         "play_segments": segments,
828     }
829     return response

```

32. assistant (2026-07-06T17:28:48.892Z)

The in-process API path already handled the Failure branch (line 806-816) correctly. This confirms the pre-existing contract.

Now I have all the pieces to reason about interactions carefully. Let me focus on the most consequential concern the task flagged: the interaction between `\_maybe\_dispatch\_remote` and `\_cached\_process\_result` / `\_probe\_process\_cache` on re-submit of the same md5 after a failed report was written.

The key question: On the cloud path, after a failed run writes `status="failed"` report.json at `outputs/<md5>/report.json`, the marker is `rc=3`, so `run\_infer` returns `ComputeResult(returncode=3)`, and orchestration line 571 ? `\_failed(...)` . Good ? no ingest of the failed report happens.

But there's a subtle **idempotency/staleness bug** in the resume path. Let me trace `\_resume\_one\_remote\_job` for a re-dispatched run. Consider: a genuine 0-rally video was submitted, timed out (wrote failed report), the user re-submits (force=false, same md5). The new cloud run is dispatched. Now the control plane restarts mid-run. `\_resume\_interrupted\_remote\_jobs` re-arms `\_resume\_one\_remote\_job`, which polls for `outputs/<md5>/report.json` to EXIST. **The OLD failed report from the first run already exists** at that path. So `poll\_until` immediately returns (the report exists), and it reads the OLD `status="failed"` ? marks the job failed ? even though the NEW in-flight run may still be computing and about to succeed. Let me verify this is a real regression vs prior behavior.

Let me check `\_read\_marker` and how `storage.exists` / the report get overwritten, and confirm the resume poll uses only existence (not freshness). Let me also verify the ordering: in `\_serialize\_and\_push\_outputs`, files are pushed sorted alphabetically ? `intervals.csv` before `report.json`. Let me confirm the sort order and the marker-after-report ordering in `\_fail`.

33. user (2026-07-06T17:28:49.061Z)

```

380     ``?video=<md5>`` lands on grounded state after any restart/redeploy wipe. True ? the
DB
381     has the video's rows now (re-query). Non-md5 ids return False without touching the

```

```

382         bucket; every failure degrades to False (the read just reports what the DB has)."""
383         vid = (video_id or "").strip().lower()
384         if not _MD5_HEX_RE.fullmatch(vid):
385             return False
386         if not probe_process_cache(db, cfg, vid):
387             return False
388         return cached_process_result(db, cfg, vid, "") is not None
389
390
391     def _maybe_dispatch_remote(
392         cfg,
393         db,
394         req: "ProcessRequest",
395         video_id: str,
396         seg_name: str,
397         val_results,
398         play_wm: int,
399         cand_wm: int,
400         notify,
401     ) -> Optional[Tuple[Dict[str, Any], bool]]:
402         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
403         ``deployment.compute_target
404         == "cloud_run"`` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
405         intervals into
406         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
407
408         Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
409         the cloud job
410         actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
411         observability report
412         doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
413         process segmenter
414         (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
415         non-``cloud_run`` target, so
416         the in-process path stays byte-identical)."""
417         from backend.compute import ComputeJobSpec, get_compute_backend
418
419     def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
420         # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
421         (id > play_wm);
422         # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
423         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
424         # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
425         only when the
426         # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
427         rc / ingest
428         # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
429         records the
430         # intended backend, `requested` the picker choice, `dispatched` whether the
431         remote job ran.
432         return (
433             {
434                 "video_id": video_id,
435                 "status": "failed",

```

34. assistant (2026-07-06T17:29:04.364Z)

Now let me examine the resume-path staleness concern more rigorously. The critical question: was this pre-existing, or is it a NEW regression introduced by the fix?

Before the fix: a detector timeout/abort returned bare ``[]`` ? the OLD code wrote a
 ``status="success"`` report.json with 0 segments (per the incident description ? "reported a run
 as status=success with 0 segments even when the detector TIMED OUT"). So even BEFORE the fix,
 ``outputs/<md5>/report.json`` existed after a timed-out run.

So the resume-poll "existence" staleness was already present before this fix ? the report already existed. The fix changes the `*status*` written (success?failed), not whether the file exists. So the resume staleness is pre-existing, not introduced. However, let me check whether there's a genuinely NEW issue: a validation failure or unknown-segmenter failure now writes a `report.json` where `**previously it wrote NONE**`. This is explicitly stated in the task: "previously validation/unknown-seg failures wrote NO `report.json` at all."

This IS a new behavior. Let me trace what a validation failure's newly-written failed report does to consumers. Let me look at ``_ingest_remote_segments`` and the resume path interaction with force, and check whether ``probe_process_cache`` on the appliance/local path is affected. Let me check the resume path when the FIRST run was a validation failure and the same md5 is re-submitted.

The scenario that matters most: `**the cloud worker's validation failure now writes `outputs/<md5>/report.json` with `status="failed"`.**` Previously it wrote nothing. Now consider ``_resume_one_remote_job`` after a restart for a DIFFERENT, later cloud run of the same md5. Actually the more direct concern: the fix writes a failed report for a validation failure at the shared ``outputs/<md5>/`` prefix. Let me verify: does the cloud worker even run validation, and would a validation-failed report now poison anything? Let me check the done marker semantics and whether ``run_infer`` even reads report on `rc!=0`.

In ``cloud_run.run_infer``: it ALWAYS reads ``report.json`` (line 236) regardless of `rc`, then returns it in ``ComputeResult.report``. Then orchestration line 571 rejects on `rc != 0` before using the report. So the failed report isn't ingested. Good.

Now the crux ? let me verify the exact `**resume-path regression with the newly-written failed report on validation failure**`, and whether force re-submit is protected. Let me re-read the resume force carve-out and the SQL that selects interrupted jobs.

35. user (2026-07-06T17:29:06.788Z)

```
51:      # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
364:      # is what enforces a single served model).
450:      # v8: record the per-request COMPUTE choice the SPA compute-picker sent ("local"
forces the
451:      # in-process segmenter; "cloud" forces a Cloud Run dispatch). It must survive the
async
```

36. user (2026-07-06T17:29:13.914Z)

No matches found

37. user (2026-07-06T17:29:16.780Z)

Found 2 files
backend\infrastructure\database_jobs.py
backend\api\job_worker.py

38. user (2026-07-06T17:29:19.956Z)

```
628:  def interrupted_remote_process_jobs(self) -> List[Dict[str, Any]]:
629:      """The RUNNING md5-keyed process jobs a restart left behind ? the rows
630:      ``recover_stale_jobs`` deliberately did NOT fail (CP-rebuild P3). Called once at
631:      startup, BEFORE the drain runs, so every 'running' process row here is orphaned by
632:      definition (the executor is in-process; no live worker thread can exist yet)."""
633:      try:
634:          with self._connect() as conn:
635:              rows = conn.execute(
636:                  "SELECT id, video_id, video_path, owner_id, compute FROM jobs "
637:                  "WHERE status = 'running' AND (kind IS NULL OR kind != 'compile') "
638:                  "AND video_id IS NOT NULL AND video_id != ' ' "
639:                  "AND COALESCE(json_extract(params, '$.force'), 0) != 1 "
640:                  "AND (compute IS NULL OR LOWER(TRIM(compute)) != 'local')")
641:              ).fetchall()
```

```

642-         return [dict(r) for r in rows]
643-     except Exception as e:
644-         logger.error(f"Failed to list interrupted remote process jobs: {e}")
645-         return []
646-

```

39. assistant (2026-07-06T17:29:36.040Z)

This confirms the resume path excludes force=true rows at the SQL level, plus the defense-in-depth in `\_resume\_one\_remote\_job`.

Now, the resume-path staleness I identified: the poll waits for `report.json` to EXIST, and an OLD report from a prior run of the same md5 already existed even before this fix (the false-success case). So the pre-existing staleness isn't newly introduced. But the fix does subtly interact: previously a **validation-failed** cloud run wrote NO report. Now it writes one. So consider a specific new poisoning scenario for the resume path:

A cloud job for md5 X ran, **validation failed**, wrote `outputs/X/report.json` (status="failed") ? this is the NEW artifact. Now the user re-submits X (fixed video, same md5? No ? a fixed video would have a different md5). So a validation failure that writes a failed report at `outputs/<md5>/` cannot be "unblocked" by re-submitting the same bytes (same md5 = same validation failure). And a genuinely different video has a different md5. So the validation-failure report doesn't poison a legit re-submit. Good ? this is safe.

Let me now zero in on the one genuinely concerning interaction I can ground concretely: the **resume-path completing a job from a STALE report of a DIFFERENT prior run**. Let me verify whether a re-submit of the same md5 (force=false) that was previously a detector-timeout can now be dispatched to the cloud AND then, on restart, resume from the OLD failed report. Let me check the submit-time cache probe gating in main.py's process endpoint to see if a failed report short-circuits re-dispatch.

40. user (2026-07-06T17:29:36.249Z)

```

905     def process_video(req: ProcessRequest, request: Request):
906         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
907
908         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
909         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
910         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
911         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
912         """
913         # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
914         # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
915         # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
916         try:
917             owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
918         except edge_auth.EdgeAuthError as e:
919             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
920
921         allowed_root = getattr(
922             getattr(request.app.state.config, "security", None), "allowed_video_root", None
923         )
924         # In hosted mode a bucket URI is allowed only when it's under the configured
input_root (the
925         # designated bucket) ? not an arbitrary bucket the runtime SA happens to reach
(tenant isolation).
926         allowed_uri_root = getattr(
927             getattr(request.app.state.config, "storage", None), "input_root", None
928         )
929         try:
930             validate_input_path(
931                 req.video_path, allowed_root=allowed_root, allowed_uri_root=allowed_uri_root

```

```

932         )
933         # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
934         # scheme, which we map to a clean 400 (an unknown scheme must never 500).
935         input_ref = storage.resolve_input_ref(
936             req.video_path, getattr(request.app.state.config, "storage", None)
937         )
938     except ValueError as e:
939         raise HTTPException(status_code=400, detail=str(e)) from e
940     # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
941     # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
942     # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
943     # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
944     # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
945     # rejects a non-local URI as out-of-root.
946     if input_ref.backend == "local" and not os.path.exists(input_ref.key):
947         raise HTTPException(
948             status_code=404, detail=f"Video file not found at '{req.video_path}'"
949         )
950     # The P2 (D11) remote-source scratch guard now lives INSIDE jobs.submit_process_job,
AFTER
951     # the cache probe (#492 review): a cached gs:// result never fetches to local
scratch, so
952     # low temp space must not 507 it; a cache MISS still fails fast pre-claim exactly as
before.
953     if req.segmenter and not segmenter_registry.get(req.segmenter):
954         available = list(segmenter_registry.list_available().keys())
955         raise HTTPException(
956             status_code=400,
957             detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}",
958         )
959
960     # ATOMIC idempotent submit (#448) + submit-time md5 cache (#484): one hook claims-
or-dedups,
961     # and completes the job INSTANTLY when this video's detection already exists in the
DB or the
962     # bucket (no drain, no GPU dispatch). `force` opts out. Logic in backend.api.jobs.
963     job_id = uuid.uuid4().hex
964     outcome = jobs_api.submit_process_job(
965         request.app.state.db, request.app.state.config, job_id, req, owner_id, input_ref
966     )
967     if outcome is None:
968         raise HTTPException(status_code=500, detail="Failed to persist job record.")
969     queued, response = outcome
970     if queued:
971         _get_executor().submit(
972             _drain_due_jobs, request.app.state.config, request.app.state.db
973         )
974     return response
975
976
977     @app.get("/api/jobs/{job_id}/cost")
978     def get_job_cost(job_id: str, request: Request):
979         """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
980         GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
981         null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
982         cost = request.app.state.db.get_job_cost(job_id)

```

```
983         if cost is None:
984             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
```

41. assistant (2026-07-06T17:29:43.875Z)

Let me check ``submit_process_job`` ? this is where the cache probe (``probe_process_cache`` / ``cached_process_result``) gates whether to dispatch. This is the key interaction for "does a failed report block/mask a re-submit."

42. user (2026-07-06T17:29:45.856Z)

```
37:     ``req.force`` opts out (deliberate reprocess). Keyed on (owner_id, video_path) ? the
input isn't
38-     hashed until the worker runs, so the submitted ref is the stable pre-flight key.
``video_id`` is
39-     the submit-time md5 when cheaply known (#484) ? persisted at CREATE so a restarted
control plane
40-     can re-associate the job with its bucket outputs (CP-rebuild $4.2); "" ? the drain fills
it.""
--
50:         force=req.force,
51:         video_id=video_id,
52:     )
53-     if claimed is None:
--
80: def submit_process_job(
81-     db: Any, cfg: Any, job_id: str, req: Any, owner_id: Optional[str], input_ref: Any
82-) -> Optional[Tuple[bool, JSONResponse]]:
83-     """The whole /api/process submit decision (#448 dedup + #484 submit-time cache) in one
hook.
--
91:     4. Only the claim WINNER realizes the hit (`cached_process_result` ? itself serialized
92-     per video_id) and completes the job instantly: no drain, no GPU dispatch.
93-
94-     Returns `(queued, response)`: `queued` True ? the caller kicks the drain; `None` ?
--
99:     probed = bool(video_id) and not req.force and orchestration.probe_process_cache(
100-         db, cfg, video_id
101-     )
102-     if input_ref.backend != "local" and not probed:
--
112:         cached = orchestration.cached_process_result(
113-             db, cfg, video_id, req.video_path, requested_compute=req.compute
114-         )
115-         if cached is not None:
```

43. assistant (2026-07-06T17:29:55.634Z)

This confirms: ``probe_process_cache`` returns True only if a ``status=="success"`` report exists (line 280 in orchestration). A ``status="failed"`` report ? ``probed=False`` ? the job is NOT short-circuited; it dispatches normally. And ``cached_process_result`` returns ``None`` on a failed report ? no false cache hit. So a failed report does NOT block or mask a legit re-submit. This is correct behavior.

I've now verified all the consumers thoroughly. Let me do a final focused check on the one thing that could be a genuine defect: the ordering/consistency in ``_fail``. Specifically, in ``_fail``, ``_serialize_and_push_outputs`` is called with ``str(getattr(args, "video_uri", ""))` or ``""`` for the failed report's ``video_uri`` field. But note: in the validation-failure and unknown-segmenter cases, ``_fail(2)`` runs, and the report is written for the same ``video_id``. Let me verify one concrete correctness concern: is ``video_id`` always defined when ``_fail`` runs? Yes ? ``_fail`` is defined after ``video_id = compute_video_id(local_video)`` (line 260), and all ``_fail`` call sites are after that. Good.

Let me verify one more potential issue: the parallel ``yolo_hybrid.py`` segmenter ? does it still

return bare `` on a detector timeout, leaving a silent-failure leak that the fix's own rationale claims to close? This would be an inconsistency but let me check if it's in scope (it's a "PARALLEL hybrid segmenter" per the task).

44. user (2026-07-06T17:29:57.130Z)

```
347:         return []
459:     def process_video(
471:         return []
482:         return []
```

45. user (2026-07-06T17:30:00.471Z)

```
459     def process_video(
460         self,
461         video_id: str,
462         video_path: str,
463         player_pool: List[str] = None,
464         max_frames: int = None,
465     ) -> List[Dict[str, Any]]:
466         self.person.lazy_load_model()
467
468         # Phase 0: resolve sport profile + AI provider (early-returns [] on any
failure).
469         provider = self._resolve_provider(video_path)
470         if provider is None:
471             return []
472
473         # Re-read the config values needed downstream (pure deterministic reads).
474         idx_cfg = self.config.indexing
475         sport_profile = sport_registry.get(self.config.sport)
476         provider_name = self.config.ai_provider
477         model_name = self.config.ai_model
478
479         video_duration = get_video_duration(video_path)
480         if video_duration <= 0:
481             logger.error("Invalid video duration.")
482             return []
483
484         chunk_duration = idx_cfg.chunk_duration
485         chunk_overlap = idx_cfg.chunk_overlap
486         velocity_thresh = (
487             idx_cfg.yolo_velocity_threshold
488         ) # Fraction of frame width per second
489         merge_gap = idx_cfg.dead_time_merge_gap
490         frame_skip = idx_cfg.yolo_frame_skip
491         tracker_config = idx_cfg.yolo_tracker
492         handoff_padding = idx_cfg.ai_handoff_padding
493         ai_max_workers = idx_cfg.ai_max_workers
494         log_candidates = idx_cfg.log_yolo_candidates
495         store_candidates = idx_cfg.store_yolo_candidates
496
497         # Snapshot of the params that produced these detections ? stored alongside each
498         # candidate so a fine-tuning run can reproduce / correlate against them.
```

46. assistant (2026-07-06T17:30:18.227Z)

`yolo\_hybrid.py` still returns bare `` on provider-resolution failure and invalid-duration (lines 471, 482). This is a DIFFERENT segmenter with the SAME silent-failure class the fix is meant to close ? but the wf294 incident is specifically about the WASB/trajectory shuttle detector timeout, and yolo_hybrid is out of the diff's scope. This is a pre-existing gap (a `provider is None` early-return is arguably a config problem, not a detector timeout), not a regression introduced by this change. It's a consistency observation, not a defect in the change under review. I won't report it as a defect of this fix (the task explicitly says report only

concrete defects in the change, not speculative gaps elsewhere).

I've now completed a thorough cross-check of every consumer:

1. `orchestration.probe_process_cache` (line 280) / `cached_process_result` (line 326)**: both gate strictly on status == "success". A "failed" report ? cache miss ? normal re-dispatch. No false hydration, no blocking of a legit re-submit (confirmed via jobs.submit_process_job` line 99).`

2. `api/job_worker._resume_one_remote_job`**: waits for report to EXIST, then cached_process_result` returns None on a failed report ? reads status ? mark_job_failed`. Strictly better than a hang. The report-existence staleness is pre-existing (a false-success report already existed before the fix), not introduced here.`

3. `compute/cloud_run.run_infer`**: rc=3 marker ? ComputeResult(returncode=3)`; orchestration line 571 rejects on rc != 0 before touching the report. Correct.`

4. `Idempotency/ordering**`: `_serialize_and_push_outputs` pushes intervals.csv` then report.json` (alphabetical); the done-marker is written AFTER the report push in _fail`. If the report push throws, it's caught and the marker still writes rc=3 ? which is the correct consistent state (a rc=3 marker with a possibly-missing report; cloud_run._read_json` returns {}` on a missing report, and orchestration still rejects on rc != 0). Consistent.`

5. `The in-process paths` (main.py` CLI line 1557, orchestration._execute_processing` line 806) already handled Failure` ? status="failed"` correctly and predate this change; the fix aligns the worker/cloud path with them.`

I found no concrete defect, regression, idempotency/concurrency bug, broken assertion, or convention violation grounded in the actual code. The fix is correct and consistent across all consumers.

47. user (2026-07-06T17:30:18.235Z)

Structured output provided successfully